Roll Drauf Creation Engine

Research and Technical Specification v1.2 — reconciled reading edition

Date: 25 September 2026. **Project phase:** G1 Deploy remains authorized. **Active implementation contract:** specification v1.1.4, core wire 1.1.3 and approved registry addendum A1. **Document status:** consolidated approved body plus explicitly scoped research changes; not a replacement implementation authorization or a wire-version upgrade.

This edition corrects the earlier v1.2 rewrite, which was based on v1.1 and omitted approved audit corrections. The earlier rewrite and its seven-command 1.2 schema are withdrawn from authority. Existing implementation continues under its approved contract. G0 remains passed, WP1–WP4 evidence remains valid for its recorded scope, and WP5 can continue without restarting Discover or Apply.

Sections 1–25 preserve the approved v1.1.4 body, apart from three listed editorial updates to historical status/provenance. The existing normative clauses, G0 closures, T01–T56, 20 requirement mappings and gate assignments are retained. Section 26 contains research; §27 relates clean coding to the adopted architecture; §28 records bounded future changes; §29 adds proposed T57–T74 and six proposed requirements; §30 identifies the active contract and next implementation step. Proposal language in §§27–29 does not amend the approved body by proximity.

The accepted single-presence product direction is retained for spatial mechanics. Detailed geometry, commands, lock fields and retry defaults do not silently become G1 requirements. The approved E4 rule still permits ordinary token movement on maps without declared spatial mechanics.

Companions: `creation-engine-v1.2-contracts.schema.json` is an exact copy of approved core wire **1.1.3**, despite the reading-edition filename. `creation-engine-v1.1-registry-a1.schema.json` and its approved addendum retain 24 additional commands. Together they define 27 approved command shapes. The acceptance catalogue separates the unchanged 56 approved cases from 18 research proposals. Historical catalogue status is not the live test dashboard; recorded implementation results are in the G1 program and §30.

The creator decides what content means. The platform translates supported intent into explicit definitions; validates, versions, executes, explains, and renders those definitions; and preserves the distinction between creative decisions and technical interpretation.

Contents

1. Normative authority and review boundary 3
2. Product contract and full target scope 3
3. Research synthesis and confidence 4
4. Identity, scope, revisions, and ownership....5
5. VTT integration and migration contract....6
6. Canonical bytes, types, and schema evolution....7
7. Rules primitives and deterministic evaluation 8
8. Runtime authority, scopes, and command admission....9
9. Activity state machine, atomic checkpoints, and randomness....10
10. Human decisions, reactions, deadlines, and event delivery 11
11. Time, duration, cadence, and event causality....13
12. Execution locks, updates, retention, and reproducibility 13
13. Choices, progression, grant multiplicity, and undo....14
14. Authorization, information flow, and View As 15
15. Guided compiler and authoring decisions....16
16. World graph, NPCs, settlements and spatial behavior....18
17. APIs, packages, imports and public extension boundaries....18
18. Resource budgets, availability, and operational targets....20
19. Rendering, publication, accessibility and diagnostics....21
20. Audit, provenance, telemetry and privacy of operations 22
21. Adversarial acceptance catalogue....22
22. Delivery sequence and evidence gates 25
23. Requirements, finding closure and proof obligations 26
24. Worked conformance traces 28
25. Source register and provenance....29
26. Architecture research incorporated after the approved baseline 30
27. Clean coding and the implementation already underway 33
28. Research changes and their contract boundaries 35
29. Acceptance reconciliation and change disposition 40
30. Active contract, implementation status and next action 43

1. Normative authority and review boundary

MUST and MUST NOT are conformance requirements. SHOULD requires a documented exception and a test demonstrating why the exception preserves the affected invariant. MAY indicates optional functionality. Descriptive research does not impose behavior. The normative body, its requirement catalogue, and companion schemas MUST agree; a discovered contradiction blocks conformance for that feature. No implementation may silently choose the weaker interpretation. Examples illustrate the body and are invalid if they conflict with it.

The requirement catalogue maps authoritative sections to acceptance scenario IDs. The schemas define the wire syntax of the selected core contracts; the body additionally defines relational, authorization, state-transition, and transaction constraints that JSON Schema cannot express. Passing schema validation alone never authorizes execution. Unspecified executable behavior is unsupported and MUST fail closed or remain explicitly manual. An extension cannot make an undefined behavior valid by inventing an unregistered enum or key.

Current authority: G0 passed for v1.1.4 and G1 implementation was authorized on 24 September 2026. The approved G1 plan supplies the concrete dependency, persistence and integration choices for that slice. Approval does not imply a production cutover or public launch. Choices for other slices remain subject to their own handoff. Their selection MUST satisfy the observable semantics here. No choice of database is allowed to weaken atomicity or authorization.

The historical VTT observations below were gathered at HEAD `94ff1b9e9c07e048151469195f262fda1d80536b`. The approved G1 plan subsequently refreshed the integration inventory, and §30 records the implementation snapshot at reconciliation. These are dated integration observations, not a current production inventory. The application uses local accounts and campaign roles; Discord is an optional linked identity. Workspace membership MUST NOT be inferred from a Discord server ID alone.

2. Product contract and full target scope

The target includes guided homebrew authoring; structured and advanced editing; reusable rules/content packages; character creation and progression; live VTT activities and effects; NPCs, lore, settlements, maps, relationships and timelines; semantic presentation and publishing; controlled imports/exports; and an eventual trusted extension ecosystem.

All authoring surfaces MUST emit the same authorized semantic commands. They MUST NOT maintain different mechanical rules implementations. The server's committed canonical state is authoritative. Client calculations are previews labelled with their input revision; they cannot debit resources or establish a roll outcome.

Creative assistance and prose import produce proposals. They MUST NOT bypass confirmation, content validation, entitlement checks, or the command boundary. The deterministic interview can operate without an LLM; this specification makes no promise to understand arbitrary prose deterministically.

Manual play is a supported operating mode. A manually adjudicated action records its declared outcome, authorized adjudicator, affected values, and reason. The UI MUST distinguish it from an outcome established by the rules evaluator. Manual fallback cannot grant a player otherwise unavailable mutation privileges or convert an unreviewed package into executable content.

Official, publisher, homebrew, and imported content use the same runtime contracts when execution and rights permit. Their provenance, entitlement, and redistribution permissions remain distinct. The system MUST NOT equate technical importability with permission to redistribute.

Capability profile	Required behavior	Excluded from that profile
`core-safe-1`	Typed acyclic formulas; declarative effects; bounded activities; resources; authorized human predicates; immutable revisions; locked dependencies; safe rendering	Arbitrary code, fixed-point evaluation, external network calls from rules
`progression-1`	Choice/grant ledger; reversible derived entitlements; explicit conflicts for irreversible consequences	Pretending past gameplay can be erased by removing a level
`world-1`	Campaign occurrences, typed scoped relationships, permission-aware mentions/ transclusion, spatial links	Automatic population/economy simulation unless separately declared
`interop-1`	Canonical lossless authorized export; previewed imports; adapter capability/loss reports	Implicit lossless round trips through other platforms
Future executable extension profile	Separately versioned isolation, capability, reproducibility, and resource contracts	Third-party script execution under `core-safe-1`

These profiles sequence delivery; they do not remove future target capabilities. Unsupported mechanics remain authored and renderable at manual quality. The UI must explain the exact automation boundary rather than imply full support.

3. Research synthesis and confidence

Evidence classes: **A** = primary technical docs/source; **B** = primary product behavior or architecture description whose implementation is not inspectable; **C** = secondary technical guide or community account; **D** = unsupported marketing claim. C may generate a hypothesis or usability question; it cannot independently establish a core architecture requirement. Claims about usefulness are design inferences, not facts established by the existence of another product.

Reference	Evidence and retained lesson	Limitation
Foundry D&D5e	A; activities, effects, advancement, and area behavior are separately modelled [R01–R05]	Similar vocabulary does not establish identical execution or migration semantics
Dungeon Master's Vault	A; choices and modifiers derive entity attributes through dependency ordering [R06]	The composition and concurrency requirements of this VTT remain ours to specify
DiceCloud	C for the guides cited by v1.0; slots, libraries, and rulesets motivate reusable choices [R07–R08]	Primary implementation corroboration is required before elevating those guides to A
Roll20	B; custom options, future-level planning, contextual queries, and attunement [R09–R12, R29]	A workflow demonstration is not a specification of its internal engine
D&D Beyond	B; composable data and a cloud ECS direction [R13–R14]	The August 2026 update describes early internal testing; it does not prove mature public homebrew capability
Fantasy Grounds	A; duration, targeting, and expenditure are distinct [R15]	Similar names such as once-per-turn can embody different reset semantics

Reference	Evidence and retained lesson	Limitation
World Anvil	B; typed relationships and optional templates [R16–R19]	Useful world-authoring reference, not proof that definitions should own campaign state
Kanka	A for checked permission documentation; semantic references and member-view testing [R20–R23]	Permission semantics must be explicitly integrated with our derived data and commands
LegendKeeper	B; low-friction creation and linked spatial knowledge [R24]	No claim that every rich world entity should be mandatory at creation
Homebrewery / publication tools	A/B; renderer generations, semantic layout, and sanitization work [R25–R26]	Historical sanitization fixes do not demonstrate safety of our renderer
Monsterbrew / Tetra-Cube	B/C pending targeted revalidation; live diagnostic and statblock interactions [R27–R28]	Advisory feedback is not an automatic balance authority

The v1.0 market-gap thesis remains a hypothesis: users may benefit from expressive mechanics behind an understandable interview. It MUST be tested with representative creation tasks. Competitor roadmaps alone MUST NOT set a launch deadline or justify bypassing correctness gates. The complete source register is retained in §25, with checked and inherited references distinguished.

4. Identity, scope, revisions, and ownership

The system MUST distinguish these records:

Record	Identity and owner	Mutable state
Definition	Stable logical content ID within an owning workspace/publisher	Name/index pointer and administrative metadata; never the bytes of a published revision
Revision	Immutable revision ID and canonical content digest	None; superseding content creates another revision
Draft	Stable draft ID, owner workspace, monotonically increasing draft version	Proposed definition and unanswered decisions
EntityInstance	Stable occurrence ID, owning runtime scope, pinned revision, optional world identity	Runtime resources, equipment state, decisions, explicitly permitted overlays
WorldEntity	Stable identity within one world scope	World facts and links, each governed by version and access policy
Representation	Token, sheet, map pin, alias, or publication reference to an occurrence/world identity	Presentation state such as map coordinates or display configuration
PackageVersion	Package ID plus immutable version and manifest digest	None; withdrawal is separate registry metadata
ExecutionLock	Immutable resolved content and evaluator closure	None; replacement is an explicit campaign migration

An ID is an opaque ASCII string of 1–128 characters using letters, digits, `.`, `_`, `:`, and `-`. Its text is never authorization. Logical IDs are globally unique within their registered namespace; import collisions MUST be resolved by verified identity or an explicit remap, never overwritten. IDs are not reused after deletion. Human names are mutable labels. Semver labels never substitute for a content digest or immutable revision identity.

A revision contains typed, namespaced components; activity, effect, resource, choice and advancement definitions; semantic presentation bindings; and source metadata. All executable references inside a published revision resolve through its ExecutionLock. Instance overlays may change only paths declared overrideable by the pinned schema and permitted by the campaign. Overrides are versioned commands with reason and provenance; arbitrary patches are forbidden.

`DefinitionRef`, `RevisionRef`, `InstanceRef`, and `WorldRef` are distinct tagged wire types. Every wire field that references an occurrence is an `InstanceRef`; a bare ID is never accepted as an occurrence reference. A relationship schema declares accepted source/target reference kinds, entity capabilities, cardinality, temporal validity and inverse behavior. Content-template relations refer to definitions/revisions; campaign relations refer to instances/world entities. Runtime lookup MUST NOT automatically substitute one kind for another.

A named NPC package supplies a reusable definition. Installing it in campaigns A and B creates independent world identities and instances. Two tokens representing the same individual in A reference the same occurrence; their position can differ, but their shared resources cannot diverge. An intentionally independent duplicate receives a new occurrence. A disguise is a separately permissioned representation, not another automatically independent HP pool.

Publications and packages MUST NOT contain campaign occurrences or secrets implicitly. A user can explicitly fork permitted content into a new definition; provenance records the origin without retaining unauthorized read access. Cross-world live mechanical links are unsupported in the baseline. Cross-campaign transfers require an explicit export/import or later transfer protocol; they cannot participate in one baseline action transaction.

Deleting a live reference target requires impact analysis. Soft-deleted objects leave a tombstone with no unauthorized name or metadata. Referenced immutable revisions remain retained under the applicable retention/rights policy. Hard deletion that makes replay impossible must record that limitation and remove the exact-replay claim; it must not silently bind the ID to replacement content.

5. VTT integration and migration contract

The first integration target is existing Roll Drauf Light characters. The first full reference ruleset is separately versioned D&D 5e, with 2014 and 2024-compatible variants explicitly distinguished. Neither is silently applied to Roll Drauf Light. The first slice uses a deliberately small test ruleset plus a real legacy-character adapter; passing it is not a claim of complete D&D coverage. A campaign selects one ruleset composition and one ExecutionLock. Mixed editions require a named compatibility profile and fixtures; otherwise combination is rejected.

The existing VTT's integer character/equipment/token IDs are mapped to new opaque IDs in a bijective migration ledger keyed by source type, source ID, and migration version. The original identifiers remain available for compatibility routing. Local account identities remain authoritative authentication principals. Campaign role, profile tier, platform staff role, and workspace membership are independent checks. For migrated VTT data the default policy grants: the character's owning account `controls` that character's occurrence; accounts holding campaign role DM or CO_DM `operate` the campaign scope; token ownership grants control of that representation's presentation state only; and the owner of a personal sandbox controls its occurrences and operates it. Any other delegation is an explicit, audited policy grant (§14).

Current persisted data	Authority after explicit migration	Legacy-write behavior
AC, proficiency, maximum resources and similar calculated values	Projection from bases, choices, grants and effects; manual overrides are explicit inputs	Translate to an authorized override command or reject with `LEGACY_FIELD_READ_ONLY`; never write projection columns directly
Current HP, charges, mana and other current resources	Runtime resource ledger	Translate to a resource adjustment under the same commit protocol
Ability scores	Declared base plus progression/effect contributions	Translate to base edit or explicit override according to the field contract

Current persisted data	Authority after explicit migration	Legacy-write behavior
Equipment / inventory rows	Instance and ownership records, preserving quantity and custom descriptions	Translate equip, transfer, split, consume and edit commands
Character content/progression JSON	Imported snapshot plus recognized selections/grants	Recognized edits translate; unknown metadata stays inert and round-trips
Token HP copied from character	View of shared occurrence state	All HP writers invoke the same resource command; token movement remains representation state
Legacy exports	Versioned compatibility projection	Import through preview; no direct ORM mutation shortcut

Unknown historical calculations MUST NOT be reverse-engineered as invented author intent. An unexplained total becomes a clearly labelled legacy baseline or override with an imported provenance marker. Known effects must not be applied again on top of an imported total without an explicit reconciliation. Unrecognized data remains inert metadata or a manual capability; it never becomes executable because a key happens to resemble a current schema field. Unrecognized legacy data is stored in the declared inert component `legacyMetadata`: opaque JSON that no formula, selector, predicate or presentation binding can address, projected by policy like any other field, and returned unchanged (identical canonical bytes) by the legacy adapter. It is the only container for unrecognized legacy data; the rejection of schema-unknown fields on commit (§6) applies to all other content.

Migration states are `LEGACY`, `SHADOW`, `FROZEN_FOR_CUTOVER`, `ENGINE`, and `ROLLBACK_REVIEW`. Migration state is a property of the RuntimeScope: all instances in a scope change authority together, and no scope contains instances under different authorities. In LEGACY/SHADOW the legacy writer remains sole authority; shadow evaluation cannot mutate live resources, and engine evaluation in such a scope is preview-only. A migration preview lists every mapped field, unresolved discrepancy, and unsupported behavior. A scope cannot enter ENGINE until, for every migrated character, its declared parity checks pass or each intentional difference is explicitly approved by that character's authorized controller and the campaign operator. A character without a campaign migrates within its owner's personal sandbox scope. Attaching a migrated character to a campaign scope after cutover requires the transfer protocol (§4); until that protocol is specified the attempt is rejected with `SCOPE_TRANSFER_UNSUPPORTED`.

Cutover fences all writers for the affected runtime scope, captures a consistent snapshot, verifies no pending actions, commits the migration ledger and authority switch atomically, then releases the fence. Requests carry an authority epoch; a stale client receives `AUTHORITY_EPOCH_MISMATCH` plus an authorized refresh instruction. Compatibility routes are adapters, not second writers. Any denormalized display cache is labelled by authority epoch, scope state version and ExecutionLock digest.

Immediate rollback is allowed only before post-cutover engine mutations. After that, rollback requires a verified reverse migration or forward compensation plan covering every committed change. Restoring an old database snapshot over subsequent gameplay is forbidden. Unsupported reverse mappings block rollback and produce an impact report. A backup is disaster recovery evidence, not proof of lossless semantic rollback.

6. Canonical bytes, types, and schema evolution

Published content is UTF-8 JSON conforming to a registered schema version. Object hashing uses RFC 8785 JSON Canonicalization Scheme [R30], then SHA-256. Duplicate JSON keys, non-finite numbers, invalid Unicode, and schema-unknown fields are rejected on commit. Strings are preserved without silent Unicode normalization; identifier grammars are restricted as above. Semantically equivalent content is not necessarily byte-identical; hashes identify exact canonical bytes, not author intent.

A content digest (`sha256:` over canonical bytes) may be delivered only to a principal who may read all of the hashed content. Values that bind server-side state for a client, namely decision context and command outcome, are **context tokens**: opaque values that the client echoes but can neither derive nor verify. A context token is either HMAC-SHA-256 over the canonical bound content under a per-scope server secret rotated with the policy epoch, or a random 256-bit nonce bound server-side to the stored canonical hash. An unkeyed digest over content that the recipient may not read in full is never delivered, because enumerating low-entropy hidden values against it would reveal them.

Mechanical numbers use exact rational values `{numerator: integer, denominator: positive integer}` in reduced form, with zero encoded as `0/1`. Wire integers are restricted to the exact safe integer range ±9,007,199,254,740,991. Intermediate arithmetic is exact and bounded by the evaluation profile; overflow or denominator zero is an error, never NaN or saturation. Units are namespaced registered identifiers with exact conversion ratios. Incompatible units cannot be added. Currency, distance, duration, and dimensionless values are distinct types.

Rounding is an explicit typed operation: `floor`, `ceil`, `truncate`, or `nearest_ties_even`. Default arithmetic does not round. A ruleset must name the rounding point for every conversion to a discrete resource, die count, or choice cardinality. Collections are either ordered sequences or mathematical sets as declared by schema; their interpretation cannot change by consumer.

Formulas are a typed numeric AST containing literals, schema-bound reads, conditionals and calls of registered pure functions. Bounded arithmetic uses the platform's core function set: `core.add`, `core.subtract`, `core.multiply`, `core.divide`, `core.negate`, `core.min`, `core.max`, and the rounding functions `core.floor`, `core.ceil`, `core.truncate` and `core.round_nearest_ties_even`; each charges one operation per call (§18). A conditional names a registered Predicate binding; boolean composition (`and`, `or`, `not`, comparisons and `is_known`) belongs to the Predicate schema (§7), not to numeric formulas. Reads identify base or a specific earlier evaluation stage; they cannot address arbitrary object paths, permissions, credentials, filesystem/network state, wall clock, or randomness. Missing required data is an explicit unresolved value. Unknown is not false, zero, or an empty list. Rules may test `is_known`; an unresolved executable result blocks that checkpoint.

Schema registries map type ID and version to wire schema, semantic validator, allowed references, permissions, evaluator identity, migration rules, guide metadata, and renderer fallback. The registry is trusted platform/ruleset configuration. Ordinary content cannot register privileged code. Schema migrations create new bytes and record input/output digests; published revisions are never rewritten. Unsupported schema versions can be stored inert only through an explicit archival import profile.

Fixed-point evaluation is excluded from `core-safe-1`. Cycles in formula reads or stage dependencies are invalid. Supporting a fixed-point profile later requires bounded iterations, convergence criterion, tie behavior, and non-convergence fixtures. Temporal feedback is modelled as ordered state transitions, not hidden dependency cycles.

7. Rules primitives and deterministic evaluation

Each primitive has stable namespaced identity within a revision. Activity, Effect, Resource, Predicate, ChoiceSlot and Advancement remain separate schemas. Presentation bindings and world relationships use the same identity infrastructure without gaining mutation authority over rules state.

An Activity declares activation mode, target selector, ordered resolution windows, requirements, costs with spend windows, typed rolls, effect applications, bounded child activities, and declared outputs. Selectors execute against an authoritative snapshot within one runtime scope. They declare maximum targets and stable ordering. An activity cannot obtain more targets by following unrestricted graph links. Hidden legal targets can be resolved server-side, but only authorized output may be returned to the invoker.

A Resource declares unit, owner scope, maximum expression, current-value policy, adjustment bound policy, spend/recovery triggers and visibility. The adjustment bound policy is either `reject`, where a change that would cross a bound fails with `RESOURCE_BOUND_EXCEEDED` and changes nothing, or `clamp_report_delta`, where the result is clamped to the bound and the receipt reports the applied delta and the discarded remainder. It has no default; a

resource without it is not runtime-ready. Current amounts are ledger state, not formula side effects. For bounded resources, `0 <= current <= max` is invariant. A maximum decrease with excess current is a conflict unless the ruleset explicitly specifies a named conversion policy, such as preserving spent amount or clamping with a reported delta. An author cannot silently select that policy through an omitted field.

Predicates return true, false, or unresolved. AUTO uses permitted snapshot reads. Human predicates reference a question contract and authorized responder selector. Manual toggles have explicit scope, owner, expiry and version; they are not global booleans. A predicate cannot execute arbitrary script under the baseline. Missing evidence never authorizes an action by default.

Effect applications carry `applicationId`, source instance, source revision, originating action/grant, target, stack group, stage, priority, duration, cadence, visibility and typed changes. Definition IDs do not substitute for application IDs. Re-equipping or retrying a command cannot accidentally create another application of the same logical effect.

Numeric targets evaluate through fixed stages: base -> set -> add -> multiply -> clamp -> final rounding. Base values are immutable inputs during a single evaluation. SET candidates choose the highest numeric priority; equal highest-priority candidates with unequal values produce `EFFECT_CONFLICT`, rather than picking by arrival order. Equal-valued candidates coalesce while retaining all provenance. ADD values are summed; SUBTRACT is normalized to a negative ADD. MULTIPLY factors are multiplied. Lower bounds take the greatest bound; upper bounds take the least. An empty interval is `EFFECT_CONFLICT`. Rounding is the target schema's explicit final operator, not a per-client choice.

Example: base 10, ADD 2 and MULTIPLY 2 produces 24 under this profile regardless of serialization order. A mechanic that requires multiplication before addition must express a separate typed intermediate value or a different versioned evaluator profile; it cannot manipulate an undocumented priority to reorder stages. Cross-stat formulas name their dependency stage and must remain acyclic.

Stack policy is one of `all`, `unique_by_source`, `highest`, or `exclusive`. It is applied within an explicit stack-group ID and evaluation stage before numeric reduction. `unique_by_source` uses the source-instance-plus-feature identity declared by the ruleset; repeated applications from that same key coalesce only if their mechanical payloads match, otherwise conflict. `highest` requires comparable same-unit scalar values and retains all equal winning provenance. `exclusive` rejects multiple active distinct sources unless a prior explicit choice selects one. Priority does not grant permission to ignore a stack conflict.

Set-valued additions/removals are sourced membership contributions, not destructive array edits. Removing one contribution does not remove another source's grant. APPEND applies only to ordered presentation sequences with explicit order keys, never to a mechanical set. GRANT is the grant-ledger operation defined in §13. UPGRADE and DOWNGRADE from v1.0 are removed as ambiguous core operators; a ruleset must express their exact semantics through the operators above or register a separately conformed evaluator.

Incremental evaluation is an optimization. Its result, unresolved-state markers and provenance MUST equal a full recomputation for the same input snapshot, lock and evaluation profile. Implementations must compare the two in conformance fixtures. A broad dependency invalidation may legitimately recalculate an entire actor; the system must still meet the declared workload budget or report that the workload is unsupported.

8. Runtime authority, scopes, and command admission

A RuntimeScope is a campaign's authoritative mechanical state, or a personal sandbox explicitly separate from a campaign. A live instance belongs to exactly one RuntimeScope. A character used in two campaigns has distinct runtime occurrences; the system MUST NOT allow two scopes to mutate one HP/charge pool without a separately specified cross-scope protocol. Shared personal identity and source definitions do not imply shared runtime state.

Each scope has a durable monotonically increasing `stateVersion`, `authorityEpoch`, `eventSequence`, active ExecutionLock and policy version. Drafts have their own versions. Neither process-local counters nor client clocks establish any of these values. Multiple server workers must observe the same committed order. A restarted worker cannot acquire authority by reusing an old in-memory lease; writers use durable fencing epochs.

Every mutation passes authentication, current usable-account status, scope membership, operation capability, resource relationship, schema validation, semantic validation, expected-version checks, execution-profile checks, and transactional revalidation. Principal identity is derived from authenticated server context. A wire `actorInstance` reference selects the game actor, not the authenticated person; `sourceInstance` identifies the occurrence that owns the invoked activity (the actor itself for an innate activity), and costs resolve against resources owned by that source occurrence unless the activity declares another owner scope. Any delegated service identity is explicit and restricted to its declared capabilities.

Commands carry command ID, idempotency key, expected scope/draft version, authority epoch, lock digest where mechanical, and a typed payload. The command ID is a client correlation identifier. It is excluded from the request digest; a retry of the same logical command MUST reuse its idempotency key and SHOULD reuse its command ID. A receipt returns the command ID of the first admitted attempt, and duplicate detection never compares command IDs. For every mechanical command other than ResolveDecision, `expectedScopeVersion` must equal the current scope version. A ResolveDecision is bound by its expected decision version and context token; its `expectedScopeVersion` must be at least the scope version at which the decision was created and is otherwise not compared. All reads used to authorize, validate costs, select targets or compute changes belong to the transaction's consistency boundary. The baseline guarantees serializable observable outcomes within a scope; implementations may use serializable transactions or equivalent locking/version checks. Per-row optimistic checks without a scope-wide consistency argument are insufficient.

Idempotency identity is `(authenticated principal, mutation scope, idempotency key)`. The canonical request digest excludes transport-only metadata and includes operation, payload, relevant expected versions and lock. Reusing a key with another digest returns `IDEMPOTENCY_KEY_REUSE` without executing. Admission, receipt creation and committed changes are atomic. In-flight duplicates return the same operation ID/status. Terminal duplicates return the original receipt filtered under current read permissions; loss of access does not replay an old secret response. A public receipt omits the internal outcome digest, lists only event IDs the recipient may read, and never reveals how many were omitted.

Durable deduplication keys and outcome digests are retained for the scope's supported lifetime, including restored backups. Compacted histories retain non-executable tombstones sufficient to reject re-execution. Deleting a scope makes its mutation namespace permanently unavailable. Rate limits and size limits also apply to invalid or duplicate traffic, but do not cause a previously committed command to execute again. Transport ACK means accepted receipt only; success means the receipt identifies a committed checkpoint.

9. Activity state machine, atomic checkpoints, and randomness

Human decisions and reactions can separate observable steps of one activity. The protocol therefore promises atomic **checkpoints**, not an impossible database transaction held open across a person's response. Every checkpoint is all-or-nothing for its costs, state changes, grants, cadence consumption, domain events, audit linkage, receipt and notification-outbox records. Earlier visible checkpoints are durable history and are never silently rolled back because a later decision is cancelled.

The action states are `PROPOSED`, `WAITING_PRECOMMIT`, `ACTIVE`, `WAITING_DECISION`, `SUSPENDED`, `COMPLETED`, `CANCELLED`, and `FAILED_PRECOMMIT`. Allowed transitions are: PROPOSED to WAITING_PRECOMMIT, ACTIVE, CANCELLED or FAILED_PRECOMMIT; WAITING_PRECOMMIT to ACTIVE, CANCELLED or FAILED_PRECOMMIT; ACTIVE to WAITING_DECISION, SUSPENDED or COMPLETED; WAITING_DECISION to ACTIVE or SUSPENDED; SUSPENDED to ACTIVE or an operator-finalized COMPLETED outcome. CANCELLED is allowed only before the first authoritative activation/roll checkpoint. A PROPOSED action becomes CANCELLED when its invoker withdraws it or its precommit deadline passes; this costs nothing, commits no checkpoint and releases its admission count. Once a roll or cost is committed, stopping the action is a recorded resolution or compensation, not deletion or precommit cancellation.

A scope admits at most one active root mechanical action. Before activation, interviews/proposals may coexist but do not reserve authority. Activation atomically acquires the durable scope gate and revalidates the full read set. During an active root, only declared reaction/child actions, its resolution commands, and authorized operator

interventions may mutate mechanical state. Unrelated mechanical commands receive `SCOPE_BUSY` and may be retried with a new command after refresh; read/chat/draft authoring continue. The gate spans players, rests, progression, equipment changes, package updates, and token movement on maps whose pinned geometry profile declares spatial mechanics (§16). Movement on any other map is representation state (§§4–5): it is not gated and triggers no mechanics. This deliberately favors correctness over simultaneous mechanical editing; a future concurrency profile requires separate proof.

The scope gate is a durable state record, not an expiring lock that silently releases an unfinished action. Worker execution leases may expire and be reassigned with a higher fencing epoch; reassignment resumes the same action/receipts. Fencing epochs are internal and never appear on the wire; a lease reassignment never changes the scope's authority epoch. The authority epoch changes only when authority over the scope is transferred: cutover, rollback review, and restore from backup. An expired worker cannot commit with its old fence. Admission limits bound stored precommit proposals per principal and scope so an author cannot exhaust persistence by opening unanswered proposals.

Activities declare a finite ordered window sequence using registered window IDs such as declare, before-roll, after-roll, before-apply, after-apply and finish. Requirements, costs and effects name their checkpoint/window explicitly. There are no default spend/refund points. An activity missing that metadata is not runtime-ready. A cost due at activation is spent at activation; a cost due after a confirmed hit is spent in that hit-resolution checkpoint. Failed checks and invalid targets do not decide refunds implicitly; the ruleset's declared outcome branch does.

The server obtains random input and records the exact dice/raw outcomes, algorithm identity, checkpoint ID and relevant rules snapshot before exposing any result to a client. Dice are stateful roll operations, never formula reads. Sampling and its durable receipt must make any retry return the same recorded result. Uncommitted random draws are never broadcast. Clients cannot provide authoritative random results except through a separately labelled, permissioned manual-roll command. Hidden future draws are not returned in seeds or debug data. Replay consumes recorded outcomes and performs no new draws.

Within a checkpoint: verify authority and gate ownership; load a consistent state; recheck permissions and prior decisions; validate all costs and target versions; evaluate bounded pure calculations; obtain or reuse checkpoint randomness; prepare the complete mutation set; then commit changes, results, receipt, events and outbox together. No committed-result notification is sent before commit. Delivery from the outbox is at least once; consumer deduplication and snapshot recovery provide one application of each event, rather than claiming exactly-once network delivery.

Failure before a checkpoint commits changes nothing from that checkpoint. Failure after commit returns its durable outcome on retry. If a later checkpoint cannot complete because of a conflict, evaluator fault or budget violation, the action becomes SUSPENDED. Earlier checkpoints remain effective. An operator can resume from the same recorded inputs after a compatible operational recovery, or issue a recorded manual resolution/compensation. Changing evaluator/content mid-action is forbidden; if the locked evaluator is unavailable, manual resolution is explicit and provenance marks the loss of automated completion.

On-hit child activities belong to the parent action tree, identify the committed hit they modify, and consume their own declared cost checkpoint. A flame charge is not consumed merely because an attack was proposed. A child cannot manufacture a new attack result or retry its parent. A cancelled optional child preserves the parent's committed roll and other costs. Scope gate release occurs only after a terminal root outcome is committed.

10. Human decisions, reactions, deadlines, and event delivery

A PendingDecision contains action/checkpoint ID, stable question ID/version, answer schema digest, context token, decision version, eligible responder rule, target references, creation time, UTC deadline, decision window and timeout policy; whether acceptance consumes a reaction slot is a property of the slot, not of the decision. The context token (§6) binds the resolved snapshot, lock, targets, required resources, policy version, and mechanical facts relevant to the question; it reveals nothing about them. User-visible text is permission-filtered and is not the decision's identity.

A response carries decision ID, expected decision version, context token, idempotency key and typed answer. The server determines the responder from authentication. A response is accepted once, only from an eligible current principal, for an open unexpired decision in the active frame. Matching retries return the same receipt. Stale responses return `DECISION_STALE`; a fresh prompt is required after relevant context changes. Changing a prerequisite answer invalidates dependent unanswered decisions; accepted postcommit outcomes are not edited retroactively.

The baseline permits one sequentially offered responder per reaction slot. The ordered candidate list of a reaction slot is fixed when the slot is created and is part of each offered decision's context token. An operator reorder of initiative while a slot is open invalidates its open decision and restarts the slot under the new order, excluding candidates whose decline is already recorded. Candidate ordering is: ruleset window priority descending; canonical encounter initiative order if applicable; stable actor-instance ID as final tie-breaker. Priority tie-breaking determines who is offered a decision, never chooses a creative answer. A slot states whether acceptance consumes the slot or whether remaining candidates are offered in order. Reaction nesting is depth-first and resumes the suspended parent window after the child terminates. Declared re-entrancy keys and the budgets in §18 prevent self-triggered or mutual loops.

Human response timeout is 120 seconds by default, selected visibly by campaign policy within 30–600 seconds and copied into the frame at creation. Declared optional reactions may expire as a recorded decline only where the ruleset explicitly defines that result. A mandatory unknown answer suspends resolution; it must not become false or spend resources automatically. Precommit proposals expire as CANCELLED without cost. When a mandatory decision reaches its deadline, the action becomes SUSPENDED in the same commit that records the expiry, and authorized operators are notified; 600 seconds is the upper end of the policy range, not a second clock. A SUSPENDED action does not release the gate or invent an outcome. Operators can reassign an eligible adjudicator, reissue the prompt, or finish manually; each intervention is an auditable operator command. This availability tradeoff is intentional and limited to that runtime scope.

Deadlines are evaluated at response-acceptance commit using server time, never a client timestamp. Persisted UTC deadlines and a nondecreasing scope time watermark prevent clock rollback/restart from reopening an expired decision. Deadline time comes from one time authority shared by all workers and is read once per commit. While a scope has open decisions, the holder of its worker lease commits a watermark heartbeat every 30 seconds. A forward-clock anomaly exists when authority time exceeds the last heartbeat by more than the heartbeat interval plus the campaign-visible tolerance (default 30 seconds); an outage is indistinguishable from a clock step and is treated the same way. Deadlines that passed inside such a gap do not expire their decisions: the owning actions become SUSPENDED for operator review, and an operator reissues or resolves them. An anomaly never invents a mandatory answer, and a backward step is evaluated against the watermark, so it never reopens an expired decision. Reissuing a prompt is an operator command that creates a new decision version and explicit deadline; it never makes the old response valid again. Admission is limited to 20 open precommit proposals per principal and 100 per scope; proposals expire after the configured precommit deadline.

Membership/permission revocation is never delayed by the mechanical gate. It updates the policy epoch, invalidates affected decisions/subscriptions, and can suspend the action. A revoked participant receives no new privileged details. An operator override that changes mechanical context is a checkpoint inside the current frame and invalidates dependent prompts; it does not masquerade as an unrelated concurrent write.

Internal domain events carry immutable event ID, scope sequence, action/checkpoint/causation IDs, state version, lock digest and audience policy metadata. Transport serializers project the event for each currently authorized audience. Clients deduplicate by permitted event ID and use an audience-bound delivery cursor for recovery; internal sequence gaps and private causation IDs are not exposed as the public event stream. A stale/invalid cursor triggers a permitted snapshot. A reconnect negotiates an authority epoch and supported schema version before applying deltas. Old epochs force a refresh.

The outbox retries until delivery acknowledgement or an operator-visible dead-letter condition. Delivery failure never rolls back committed game state. Consumers cannot invoke a mutating action solely because an untrusted client repeated an event; downstream rules triggers are scheduled from durable domain events under the action protocol. Any eventual jobs that mutate mechanics must reacquire normal scope authority.

11. Time, duration, cadence, and event causality

Mechanical clocks are ruleset-declared logical domains. The platform provides ordered event/tick identities, not hardcoded D&D rest semantics. D&D-specific short rest, long rest, dawn, turn and round are namespaced events and counters in its ruleset. Wall-clock deadlines govern UI waiting and operational leases only; they do not advance game-world time.

An encounter has a stable encounter ID and a monotonically increasing turn-occurrence ID. Reordering initiative changes future order but never renumbers committed turn occurrences. Round identity is a separate counter advanced by a committed ruleset event. Each start/end boundary is a durable event with a unique identity. Repeated transport delivery cannot expire an effect or restore a resource twice.

Duration is either explicit logical ticks with a clock/anchor and start/end boundary, an event matcher with an occurrence count, or permanent. The duration begins at its declared application checkpoint. To avoid off-by-one interpretation, the first counted expiry boundary is strictly after the application event in scope order. A ruleset needing same-boundary expiry declares an explicit window-local duration instead. An expired source or removed target produces the declared terminal policy; it does not leave a timer attached to a reused ID.

Cadence consists of bucket selector, limit and consumption window. Bucket selectors include each-turn-occurrence, source-turn-interval, round-occurrence, encounter, next-matching-action and next-matching-roll. Source-turn-interval begins at one source turn start and ends before its next start; each-turn-occurrence permits a distinct bucket on each actor's turn. The text “once per turn” alone is invalid executable content. Outside a matching encounter/clock, a cadence must specify a supported fallback or resolve manually.

Usage identity is `(application or shared-feature key, target scope if declared, bucket ID)`. All activities sharing that feature key consume the same ledger. Consumption and its affected roll/effect commit in one checkpoint. Rejecting or retrying a command cannot increment the ledger twice. A miss consumes cadence only if the ruleset assigns consumption to an attempted action rather than a successful application.

Recovery is a resource command caused by one named logical event and deduplicated by resource, rule and event ID. Advancing world time across several dawns emits the declared number of recovery occurrences or an explicitly specified coalesced recovery event. Pausing a session never infers a rest. Combat rewind is an administrative branch/compensation; it does not decrement event IDs or erase usage history.

12. Execution locks, updates, retention, and reproducibility

An ExecutionLock records exact package versions and hashes; every transitive definition revision; ruleset ID/version/digest; schema registry bundle; formula/effect evaluator version; deterministic configuration and unit tables; extension runtime identity if any; and immutable referenced assets needed for the declared execution profile. Compatible version ranges are resolved only when creating a new lock. Execution never resolves a floating range on demand.

The registry rejects missing dependencies, ambiguous identity, digest mismatch, incompatible engines and forbidden dependency cycles before a lock becomes usable. Cyclic semantic world links can be valid; package-install dependency cycles and evaluator cycles are distinct checks. Content can extend another package only through declared typed extension points; no implicit path replacement or monkey patching is permitted.

Reproduction guarantees are separate: **mechanical replay** reproduces state/provenance from a supported snapshot and recorded outcomes under the exact lock; **semantic rendering** reproduces declared values and structure under a specified renderer; **byte-identical publication** requires retention of the rendered artifact itself and its digest. A renderer compatible range guarantees neither identical pagination nor identical bytes. Published artifacts pin exact renderer/theme/font/asset versions, locale and input projection digest; the exported artifact is retained when exact reproduction is promised.

Update workflow is preview -> impact analysis -> explicit campaign approval -> quiescent migration -> postcondition verification -> activation. The preview contains old/new locks, semantic diffs, affected instances, overridden paths, changed choices, current resource reconciliation, active effects, and unsupported migrations. Campaign channels may notify automatically, but mechanical updates never activate silently or during an active action. Text-only updates remain new revisions and cannot bypass visibility/rights checks.

Instance changes use a three-way comparison: old source revision, current instance overlay, new source revision. Overlapping incompatible changes are conflicts. Non-conflicting presentation changes may merge under declared policy. Renamed paths require explicit migration maps. Lossy migrations require an exact loss report and explicit selection; automatic semantic-version bumps are advisory metadata, not permission to migrate.

For current=2 and new max=1, an unspecified resource conversion blocks activation. A named, approved clamp records delta=-1 and its reason; a preserve-spent policy computes from the documented old maximum. Neither is silently inferred. Removing active effects or grants likewise requires explicit migration semantics.

Published bytes and lock components are retained while supported live instances or replay promises depend on them. An author withdrawing a listing removes discovery, not silently the referenced bytes. Security quarantine or rights restrictions may make continued execution unavailable: affected scopes fail closed for that content, receive an impact report, and can migrate or use explicitly authorized manual play. The platform does not promise perpetual execution of prohibited content. A restricted evaluator is never run solely to satisfy a pin.

Rollback after intervening runtime mutations requires a reverse migration over current state, not restoration of stale state. A successful update journal records prior lock, migration version, before/after digests, policy decision, resource conversions and acceptance results. Backup/restore must include deduplication tombstones, random outcomes, pending decisions, locks, outbox and policy epochs at a consistent checkpoint, and a restore increments the authority epoch of every restored scope, so retries of pre-restore commands are rejected with `AUTHORITY_EPOCH_MISMATCH` instead of executing again. Replaying an outbox after restore is safe because event IDs remain stable.

13. Choices, progression, grant multiplicity, and undo

A ChoiceSlot specifies trigger, min/max cardinality, distinctness key, allowed reference kinds/packages, filters, prerequisites, replacement policy and visibility. Options are evaluated against the campaign's lock, entitlements and current selections. Prerequisite recursion must be acyclic; a circular eligibility chain is an invalid definition. Failure to find enough eligible distinct options blocks completion with a specific diagnostic, not a random fallback.

Grant records have a unique grant ID and source occurrence: class-track/level/advancement, selected option, item application, or operator command. They reference the granted capability and its merge identity. Multiple sources of one proficiency produce one effective proficiency with multiple live contributions. Revoking one source removes only its grant record. Stackable items instead create separate occurrence IDs. Rulesets must declare which behavior a grant has; identical display names do not establish identity.

Character projection evaluates retained base decisions, live grants, permitted overrides, equipment state and runtime effects. Each class/progression track has an independent identity. Level simulation runs in an isolated ephemeral scope with copied choices and deterministic fixture inputs; it cannot spend or grant into the live character. Future-level planning stores provisional choices separately and revalidates them when the real level is attained.

Removing a level is a semantic command with an impact preview. It revokes that level's contributions, invalidates dependent selections, recomputes maxima and reports conflicts. A feat granted independently survives. A granted item that was transferred, consumed, transformed or used in later state cannot be blindly deleted or recreated. The operation must present a ruleset-supported compensation or require operator adjudication. No such unresolved conflict is auto-approved.

Draft undo restores prior draft state through a new versioned command and preserves the decision log. It does not decrement versions or erase audit history. Runtime undo distinguishes reversible local changes from compensating actions. A reversal requires proof that affected values and dependencies have not changed since the checkpoint; otherwise it returns `UNDO_CONFLICT` with an authorized impact report. Published roll information cannot be made unknown again, and a reversal cannot grant a fresh random draw for the same action.

Retcons create a named branch of history or explicit compensation in the current branch. They preserve the original event chain. Transfer ownership, action cost, condition expiration and resource recovery remain sourced events so the operator can understand what a compensation changes. The UI must not label “restore this number” as “undo the entire action.”

14. Authorization, information flow, and View As

Security policy is a versioned mutable overlay separate from immutable content. Retaining a revision for replay does not freeze a person's access forever. Every response and cached delivery uses current account status, membership, entitlement and policy. Content carries visibility intentions and required capabilities, but importing those declarations cannot create actual grants or ownership.

Policies grant named operations to explicit principal/role selectors and may restrict child fields, components, relationships, effects, decisions and representations. Two relations recur in this specification: `controls(principal, occurrence)` holds when policy grants that principal control of the occurrence, and `operates(principal, scope)` holds when policy grants the campaign-operator capability for the scope. Both are default-deny policy grants; neither is inferred from content, from ownership of a representation, or from any wire field. Default is deny. Container and child policy restrictions are intersected; a child may narrow access but cannot widen a parent's audience without an explicit authorized publication/reveal operation. Explicit denies prevail over grants. Owner-private data requires the owner's grant; GM/admin viewing capability does not automatically bypass it. Platform emergency access, if introduced, requires a separately specified audited policy and is excluded from ordinary View As.

Operation	Minimum required authority; additional object policy still applies
Read projection / explanation / search	Usable account or explicit public publication; allowed scope and field read
Edit private draft	Owner or delegated draft-editor grant
Publish package	Publisher grant plus content rights and validation gate
Deploy/update campaign content	Campaign operator and deploy capability; player authorship alone is insufficient
Invoke actor activity	Current control of that occurrence, activity invoke permission and action requirements
Resolve human predicate	Current responder eligibility for that exact PendingDecision
Change a resource manually	Explicit control/adjust permission; shared or other-player resources are not implied
Transfer ownership / reveal secret / compensate action	Explicit corresponding permission; auditable reason
Export	Read plus export/redistribution permission for every included object and dependency
Execute formula over restricted fields	Server-only engine-compute grant within the declared action; never arbitrary client access

A server evaluator may read restricted mechanical inputs only where their owner/policy authorizes that use. Derived outputs inherit the intersection of source audiences. A ruleset may declare a reviewed declassification rule for a particular outcome, such as exposing resolved damage while hiding a trap's identity. That rule must name permitted output fields, input categories and audience; an ordinary package author cannot declassify data by marking an output public. Publishing an exact final number can reveal information about its hidden inputs; that is a deliberate gameplay disclosure under the rule, not a guarantee of mathematical non-inference.

Explanations and provenance undergo the same policy projection. They may reveal permitted calculation contributions but never hidden names, source IDs, decision text, URLs or exact counts of omitted sources. The public explanation may state generically that some calculation detail is unavailable when that disclosure is permitted. It must not claim a complete proof while withholding inputs. Without an allowed declassification rule, the derived value itself is withheld or the action requires authorized manual adjudication.

The output boundary covers reads, command receipts, validation errors, conflict diffs, autocomplete, search/ranking, counts, graph layouts, WebSocket events, snapshots, notifications, PDFs, embeds, imports/exports, simulation traces, audit views, operational telemetry, and every digest or token delivered to a client (§6). Unknown and inaccessible object references produce the same public not-found representation. Internal diagnostics retain a protected reason code. Search authorization precedes candidate ranking and aggregation; inaccessible candidates never affect returned totals or suggestions under the baseline.

The baseline intentionally discloses scope-level state-version changes and busy/suspended status to authorized mechanical participants so they can coordinate actions. It does not promise to hide whether any campaign mutation occurred. This narrow operational-metadata disclosure does not authorize source names, hidden object counts, private event IDs or exact omitted-contribution counts. A campaign requiring concealment even of mutation activity needs a separately reviewed transport/concurrency profile; it cannot claim that stronger guarantee under this baseline.

Cache identity includes principal/audience-equivalence class, scope, policy epoch, entitlement epoch, input state/version, ExecutionLock and representation version. Shared-cache optimization requires proof of equivalent permissions. On revocation, subscriptions are invalidated and subsequent messages re-authorized; cached responses under old epochs are not reused. The contract cannot revoke screenshots, downloaded files or information already disclosed. Public immutable publications therefore require deliberate authorization before release; future removal disables hosting/discovery, not retroactive recall.

View As is a read-only projection computed as the intersection of the initiating operator's permitted view and the selected member's permitted view, with owner-private exceptions applied. It never grants either person's additional private rights to the other. Commands, exports with greater authority, tokens and cookies cannot be impersonated through this mode. The session identifies the operator and viewed member separately, expires after 15 minutes by default, and logs entry/exit. Any actual delegated mutation is a distinct normal command, not a View As side effect.

The legacy full `character_data` serializer cannot serve migrated field-policy data. The compatibility endpoint must call the same projection policy as new endpoints before exposing new components. Restricted metadata cannot be hidden only in the frontend. Existing role-scoped socket rooms remain a delivery mechanism; they do not substitute for field projection.

15. Guided compiler and authoring decisions

Guided mode is a deterministic schema-and-constraint interview. Structured mode organizes the same editable concepts non-linearly. Advanced mode exposes typed canonical values, references, validation, diffs and provenance. None can bypass server invariants. A registered schema's authoring metadata includes stable question IDs/version, accepted answer schema, display/help localization, required/visible conditions, dependency links, priority, adapter identity and invalidation rules. Answer adapters are registered pure transformations with versioned conformance fixtures, not content-supplied executable code.

Before natural-language import is implemented, the Guided entry point offers domain intents such as create equipment, create actor, add activity, change a value, add choice, and describe a place. Those selections are explicit user decisions. Free text can always be retained as prose; unrecognized text does not automatically populate mechanical fields. Controlled aliases may map an exact normalized term to one concept only when registered as semantically equivalent for that locale and ruleset. Mechanical ambiguities always remain proposals requiring selection.

Question selection is deterministic: compute the current constraint graph; suppress irrelevant nodes; identify unresolved required decisions for the user's selected completion profile; choose the highest registered priority, then schema order, then stable question ID. No answer is guessed merely to make progress. Constraints must not depend cyclically on unanswered decisions without an explicit discriminating question that breaks the cycle. A compiler cycle or lack of a valid next question returns `GUIDE_DEAD_END` with an authorized diagnostic and Structured-mode recovery, not an endless interview.

Every user answer is an atomic draft command carrying expected draft version. It records question/version, typed answer, author, resulting canonical mutations and invalidated descendants. Changing an earlier answer preserves superseded history but marks dependent answers as stale; their effects are removed from the active draft or surfaced as explicit conflicts, never silently retained. Switching modes preserves the graph and recomputes unresolved constraints. Stable answers are reapplied across schema upgrades only through a reviewed migration; migration cannot invent an answer.

An author can stop at any time. Drafts preserve incomplete values and unresolved questions. Preview marks unresolved values without converting them to zero. The user can skip optional questions. A required creative value—damage, duration, target count, range, rarity, recharge, scaling, cost, or equivalent—has no implicit default. Explicitly choosing a preset imports its stated values as user-approved source decisions and displays them before commit. System-generated IDs or technical sorting choices may default because they do not change intended game behavior.

For “a sword that bursts into flames three times per day,” the first mechanical question is what the flames do: visual/light only, additional damage, separate burst, transformation, or another supported/manual behavior. Damage amount is asked only if damage was selected. The author separately chooses timing, duration, cost and what “day” means. Cosmetic behavior does not acquire a damage primitive. An unsupported concept can remain prose/manual or require a registered extension; it cannot become an opaque executable escape hatch.

AI/prose import is an optional proposal producer with confidence and source-span annotations. It MUST show proposed changes and unresolved ambiguity before committing them. Accepting one proposal does not authorize unrelated generated mechanics. Creative Assist is separately enabled for the specific session/task and its proposals are marked as generated. The same command validators and no-hidden-default tests apply after proposal acceptance.

Publication and automation quality are independent axes:

Mechanical quality	Meaning
Lore-only	Identity and allowed world/presentation fields are valid; no executable mechanics promised
Sheet-ready	Manual mechanics render honestly; required references and fields for those views are valid
Runtime-ready	Each enabled activity has valid lifecycle, costs, windows and authorized resolvers; human decisions may remain
Automation-complete	Every enabled mechanical branch can resolve without ad hoc adjudication for the declared capability/workload envelope; no unsupported executable prose

Publication requires rights/source metadata, policy, references, renderer safety, declared mechanical quality and packaging validation. A lore-only package can be published. An automatically executable package can be private. A human-resolved predicate is compatible with Runtime-ready, but not with Automation-complete if human adjudication is necessary. A normal user choice between valid actions is not unresolved rule interpretation.

16. World graph, NPCs, settlements and spatial behavior

World entities begin with a title and paragraph. Optional typed components can add creature mechanics, population, government, organization, shop, faction, timeline or spatial metadata. Adding/removing a component preserves identity but validates every dependent relationship, view and mechanic. Component removal cannot silently orphan live actions, choice references or secrets; it returns an impact preview and either a supported migration or a conflict.

Relationships have their own IDs, scope, typed endpoints, attributes, validity interval, policy and version. A relationship's existence is visible only when its own policy and endpoint-disclosure rules permit it. Inaccessible endpoints are omitted by default; anonymous placeholders require an explicit allowed-existence disclosure. Inverse views are computed from one canonical edge, not separately edited duplicate edges. Cardinality constraints are transactional across competing edits.

Mentions store stable typed references plus optional author-authored display alias. Renaming the referenced entity changes its normal displayed name without altering identity; an explicit alias remains the alias. Unresolved mentions remain draft tokens until linked or created through an authorized command. Autocomplete searches accessible entities only. Creating an inline placeholder creates a scoped WorldEntity, not a globally shared publisher definition.

Transclusion is permission-checked at render time, bounded by depth and expansion limits, and cycle-detected. A public page cannot reveal a private paragraph by embedding it. Exported static pages include only the export principal's explicitly publishable projection; future permission changes do not retroactively alter already downloaded artifacts.

Settlement structure links districts, sites, factions, residents, routes, shops and encounters. “Small port town” may select a supported section to discuss but MUST NOT fabricate population, government or economy. Relationship maps, family trees, organization charts and settlement views derive from authorized canonical data. Layout caches are scoped by audience; hidden nodes must not affect visible labels or counts. Any intentionally revealed unknown relationship is a separate disclosed fact.

Maps contain representations with explicit coordinate frame, scale, units and transform. Spatial mechanics declare whether they use tokens' paths, endpoints, occupied shapes, height and teleport classification. Enter/leave events derive from committed movement under one geometry-profile version, not from browser animation frames. Movement is a gated mechanical command only on maps whose pinned geometry profile declares spatial mechanics; elsewhere it is representation state and emits no enter/leave events. Teleport never implies every region along the straight line was traversed. Area targeting and difficult terrain use the pinned ruleset's geometry profile. If none is declared, the region is presentation-only or manually adjudicated; Runtime-ready spatial automation is prohibited.

The baseline geometry profile supports 2D circles and simple polygons, no self-intersecting polygons, exact rational world coordinates, and a documented boundary rule: touching counts as intersection. A token intersects a region if its declared occupied footprint intersects it. A continuous move emits each crossed boundary in increasing path parameter, with equal-parameter ties ordered by region ID; a teleport emits exit for old-only intersections and entry for new-only intersections. Area effects apply once per application/target while intersecting unless a ruleset explicitly declares pulse cadence. Every movement-trigger chain shares the action budgets and scope gate.

17. APIs, packages, imports and public extension boundaries

Read APIs expose definitions/revisions, permitted instances/projections, schema metadata, package manifests and explanations. Command APIs expose draft changes, publishing, instantiation, activity invocation, decision responses, manual adjustments, migration and compensation. Collection ordering is deterministic by declared sort key plus stable ID; pagination cursors bind the query, audience/policy epoch and snapshot. An expired cursor causes a restart rather than mixing private results from another policy snapshot.

The companion schema defines core command envelopes, invocation and response payloads, effect applications, timing, references and locks. Feature registries add versioned payload schemas for other commands. An unknown type/version is rejected; an object-shaped `payload` is never treated as a license to accept arbitrary JSON. REST and WebSocket transports use the same command IDs, validation and receipts. Compatibility clients negotiate supported schemas and cannot request removal of safety fields.

Response condition	Stable code and HTTP interpretation
Committed checkpoint	200/201 receipt with outcome, state version and authorized projection
Admitted asynchronous or pending action	202 with operation/action ID and authorized status
Invalid wire syntax	400 `SCHEMA_INVALID`
Inaccessible/missing target	404 `NOT_FOUND` with no hidden identifier detail
Known accessible operation without permission	403 `OPERATION_FORBIDDEN`
Version, authority, decision or semantic conflict	409 with specific code and authorized diff/refresh guidance
Valid syntax but invalid executable definition	422 `SEMANTIC_INVALID`
Request body exceeds the transport size limit	413 `PAYLOAD_TOO_LARGE`
Definition or request exceeds a declared structural bound at admission	422 `BUDGET_EXCEEDED`; nothing is written
Admission limit reached (open proposals, rate)	429 `ADMISSION_LIMIT`
Deterministic budget exceeded during a checkpoint after admission	422 `BUDGET_EXCEEDED` with the action status (SUSPENDED, or FAILED_PRECOMMIT for an activation checkpoint) and the last committed checkpoint ID
Operational evaluation interruption	503 plus stable operation/checkpoint status; no false success

Package manifests identify immutable versions, ruleset/engine compatibility ranges, declared definition revisions, exact included file hashes, source/rights metadata, declared capabilities and extension points. Dependency solving happens during preview and produces an exact lock. Integrity hashes identify bytes; publisher signatures additionally authenticate origin under a separate trusted-key policy. Neither confers authorization to run privileged code. Signing-key revocation affects future trust decisions and triggers an impact report for installed content.

Canonical export is lossless only for the selected authorized export scope and its declared closure. It includes schema version, exact references, required assets, provenance permitted for export, and an explicit manifest of exclusions. If rights prohibit a required dependency, the user chooses an explicitly external-reference export or the operation fails; the platform cannot label a redacted/incomplete bundle self-contained or lossless. Secrets require explicit export/publication authority, not mere view access.

Import has upload -> parse -> validate -> resolve rights/identity -> preview -> explicit commit. Uploaded data is quarantined and inert. Preview is bound to file digest, parser/adapter version, target scope, policy epoch and mapping decisions. Changed bytes or scope require another preview. Imported owner IDs, policy grants, event

sequences, invocation receipts or runtime outcomes are not authoritative. Ordinary imports create/remap local IDs and ownership under the importer's permissions. Exact identity-preserving restore requires a separate authenticated recovery operation and consistent backup manifest.

Adapter exports identify their adapter version, target platform/schema version, supported semantics and losses by source path. A partial adapter may never silently omit a cost, predicate, duration, visibility restriction or resource link while marking an object automated. The user must explicitly accept the loss report or select a safe manual rendering. Round-trip guarantees are limited to declared fixture coverage. Future executable modules remain unavailable until an independent extension profile specifies host isolation, capability enforcement, no ambient secrets, resource budgets, lock/replay semantics and emergency revocation.

18. Resource budgets, availability, and operational targets

`core-safe-1` has deterministic work limits, enforced cumulatively across one root action including all child/reaction frames. Admission rejects definitions whose declared upper bounds exceed the profile. Runtime limits still apply where branching or queries affect actual work. Exceeding a deterministic limit is `BUDGET_EXCEEDED`; it cannot produce partial writes within the current checkpoint. Earlier checkpoints follow the suspension rules in §9. The HTTP interpretation of these outcomes is fixed in §17; 503 is reserved for operational interruption and never reports a deterministic budget failure.

Limit	Baseline value
Formula AST depth / node count per expression	64 / 4,096
Intermediate rational numerator/denominator	256 bits each; final wire values within §6 range
Derived nodes / active effect applications per instance	10,000 / 2,000
Formula primitive operations per checkpoint / root action	100,000 / 1,000,000
Targets per activity / distinct affected instances per root	64 / 128
Child frames / nesting depth per root	256 / 16
Trigger firings / generated domain events per root	1,024 / 4,096
World traversal visited nodes / transclusion depth	10,000 / 8
Expanded rendered text per interactive document	1 MiB UTF-8
Upload compressed / expanded package size	25 MiB / 100 MiB
Archive members / maximum expansion ratio	5,000 / 20:1; both size limits also apply
Migration/preview affected instances per interactive request	128; larger operations use explicit bounded jobs

Each AST literal, read, arithmetic/boolean operation and conditional evaluation charges one operation when executed; registered compound functions declare and charge their internal primitive work plus input-iteration work. Query node visits, trigger firings and event creation count against their own limits and cannot hide work inside one “function call.” A declared ruleset function without a bounded cost contract is unavailable in this profile. Runtime fuel and memory accounting must include unsuccessful branches and retries inside one evaluation, not restart the budget for each child.

Imports reject path traversal, absolute paths, symlink/device entries, duplicate paths, malformed encodings, zip bombs and references outside the allowed package closure. Baseline render/import workers have no outbound network access. Remote URLs are retained as inert links or ingested through a separate authenticated asset service with scheme/host controls, request/response size limits and no private-network fetches. HTML/CSS is never executed as server code. Operational worker isolation and memory ceilings must contain a cancelled or pathological job to that job.

The following are **acceptance targets, not measured results**. A reproducible benchmark manifest must state hardware, worker count, database, network conditions and fixture digests. Reference workload: 10 concurrent campaigns, 8 connected users each, 32 active actors per campaign, up to 1,000 derived nodes and 100 effects per actor, 10,000 world entities and 50,000 relationships per campaign. Target p95 server-side simple edit/projection latency <=200 ms; p95 no-human single-target action checkpoint <=300 ms; permitted search first page <=300 ms. Client end-to-end latency and p99/max must also be reported, not hidden by the p95 target.

At the profile's structural maxima, correctness, bounded work and explicit admission/backpressure are required; the interactive SLO above is not silently extended to every maximum-size combination. Simulations/large exports use cancellable jobs with progress and fixed input snapshots. Jobs have per-scope concurrency limits and separate worker capacity so they cannot starve live actions. Operational timeouts may abort/retry an uncommitted checkpoint but cannot change its mathematical result or regenerate a committed roll.

Recovery acceptance: worker restart must recover committed receipts and pending frames with no duplicated spend; an acknowledged checkpoint must survive the configured durable-write boundary. Disaster recovery has a declared recovery-point objective and must disclose any restored-data loss. Exactly zero disaster data loss is not claimed without an infrastructure guarantee. Restore drills verify locks, policy, scope versions, deduplication, pending decisions and outbox as one consistency set.

19. Rendering, publication, accessibility and diagnostics

Renderers consume permission-projected canonical data and a semantic presentation tree. They never establish mechanical values. Bindings reference registered value paths and typed formatting functions; they cannot execute arbitrary template expressions or access unprojected objects. Missing values render an explicit unresolved marker in drafts; published Runtime-ready views cannot silently replace them with zero or omit a mandatory cost.

The semantic tree covers headings, paragraphs, lists, tables, statblocks, activity controls, formula/value references, source links, images with alternative text, and permissioned blocks. VTT sheets, cards, tooltips, PDFs, public pages, Discord embeds and accessible JSON/text are projections of that tree. Semantic source and author-written flavor are retained separately so generated mechanical wording does not overwrite authorship.

Default themes are a validated declarative style subset. Raw script, event-handler attributes, executable URLs, unsafe embedded objects, and external resource fetches are prohibited. Rendering occurs against an already authorized projection; sanitization is additional protection, not a substitute for secrecy. PDF/public export jobs use the same projection and asset constraints as interactive rendering. Trusted publisher styling cannot mutate game state and does not bypass policy.

Published artifacts record exact renderer/theme/font/asset identities, locale, projection digest and output digest. Mechanically equivalent text in another language is a new presentation artifact. Regeneration with another renderer is explicit. Old artifacts are not silently overwritten when a theme updates.

Diagnostics report severity, affected path, cause, suggested choices and expected effect. Balance/CR/damage-per-round feedback is advisory, names its formula and assumptions, and never changes content automatically. Missing rights, unresolved execution semantics, unsafe permissions and invalid references are publication/runtime gates appropriate to the claimed quality. Accessibility acceptance includes keyboard-complete authoring, labelled controls, focus preservation during questions, text alternatives for graph views, and non-color-only error/status signals.

20. Audit, provenance, telemetry and privacy of operations

Domain events describe committed game facts; security audit records describe sensitive operations and authority; creative decisions explain content authorship. These are linked but distinct streams. Each checkpoint stores the IDs linking them transactionally. An immutable revision's provenance references its source revision, component, activity/advancement, user/compiler decision and mutation. Derived explanations are computed from that graph and permission-projected.

Operational logs record trace ID, operation ID, stable error code, duration, budget usage and versions. They do not include free text, secret names, prompt answers, authorization tokens or full package bodies by default. Any temporary diagnostic capture requires scoped access, explicit retention and redaction. Security audit access is a separate capability and does not automatically grant content export access.

Audit entries are append-only under application permissions; an administrative repair is another entry. “Append-only” does not claim immunity to a database administrator. Integrity verification and backup protections must match the declared operational threat model. Retention and deletion policy must identify which payloads can be redacted while preserving outcome/deduplication integrity, and when such deletion invalidates historical replay guarantees.

Required metrics include command conflict and deduplication rates, suspended-action age, prompt timeout, unauthorized access attempts, recomputation cost, deterministic budget failures, queue/outbox lag, migration discrepancies, compiler abandonment and adapter losses. Metric labels cannot contain user-generated secret content or unbounded IDs. Alerts distinguish one malformed package from scope-wide or fleet-wide service degradation.

21. Adversarial acceptance catalogue

These are normative future acceptance scenarios, not executed Creation Engine tests. The companion JSON records each as `specified_not_executed`. Tests use isolated fixtures, fixed content digests, controlled clocks/randomness and explicit role identities. A happy-path demo does not waive applicable negative cases.

ID	Setup / adversarial action	Required observable result
T01	Migrate legacy AC with unknown contributions	Preserve labelled legacy baseline/override; no invented provenance or double application
T02	Old client writes calculated field after cutover	Translate to allowed command or reject; no second authoritative write
T03	Two representations and two independent campaign occurrences	Same occurrence shares resources; independent campaigns do not
T04	Two commands spend last shared charge	One succeeds; current never negative
T05	Crash before checkpoint commit, then retry	No partial change or exposed uncommitted roll; one eventual outcome
T06	Crash after commit before response; retry on another worker	Same receipt, roll and debit; no extra effect

ID	Setup / adversarial action	Required observable result
T07	Reuse idempotency key with altered target/amount	IDEMPOTENCY_KEY_REUSE; no second execution
T08	Duplicate outbox delivery and lost socket delta	Apply event once; recover permitted consistent snapshot
T09	Revoke responder access while prompt is open	Reject old answer; no secret result; suspend/reassign as specified
T10	Operator changes relevant mechanical context	Invalidate prompt; stale token cannot authorize effects
T11	Optional versus mandatory decision times out	Declared optional decline or mandatory suspension; no invented answer
T12	Cancel after committed roll and retry	Preserve roll; no precommit cancellation or free reroll
T13	Reaction chain exceeds depth or cycles	Bounded checkpoint failure; retain earlier commits and suspend
T14	Permute base 10, ADD 2, MULTIPLY 2 serialization	Result 24 and equivalent provenance under core-safe-1
T15	Different SET values at equal highest priority	EFFECT_CONFLICT; arrival order cannot select winner
T16	Compare incremental with full evaluation across generated fixtures	Equal values, unresolved markers and source contributions
T17	Reorder initiative; duplicate turn/recovery events	Stable occurrence IDs; no repeated cadence/recovery
T18	Source-turn versus each-turn cadence	Distinct declared buckets behave as specified
T19	Restore pinned campaign after package/evaluator release	Replay exact lock and recorded outcomes or explicit unavailable state
T20	Max charges decreases from 3 to 1; current is 2	Block until explicit conversion; record approved exact delta
T21	Roll back package update after new gameplay	Evaluate current state; no stale snapshot overwrite
T22	Player explains/exports value influenced by hidden curse	Only approved declassified result; private metadata absent
T23	GM warms cache; player/other campaign requests same IDs	No audience/policy/scope cache crossover
T24	View As requests owner-private field and mutation	No extra private rights; refuse mutation
T25	Revoke membership mid-subscription; retry old receipt	Use current policy; no stale privileged payload
T26	Huge valid graph, trigger loop, archive bomb or external asset	Enforce declared budgets/isolation; no partial current checkpoint
T27	Same NPC installed in two campaigns; one changes allegiance	Independent state; definition/other campaign unchanged
T28	Two sources grant proficiency; revoke one	Surviving source preserves proficiency
T29	Remove level after granted item was transferred/ consumed	Conflict or accepted compensation; no blind deletion/ recreation
T30	Undo after later action changes affected resource	UNDO_CONFLICT or declared compensation; preserve later work

ID	Setup / adversarial action	Required observable result
T31	Harmless flames; change an earlier interview answer	No damage invented; invalidate dependent answers/ effects visibly
T32	Guided -> Advanced -> Guided, locale/schema change	Retain valid decisions; ask new ambiguities instead of defaulting
T33	Contradictory/cyclic interview constraints	Finite GUIDE_DEAD_END and recovery; no infinite loop
T34	Progress-clock/success-pool ruleset fixture	Use declared contracts without core rest/turn hardcoding
T35	Publish lore-only content; falsely claim automation with human-only predicate	Allow valid lore publication; reject false automation claim
T36	Import forged owner, existing instance ID and grants	Inert preview and authorized remap; no authority takeover
T37	Required export dependency is not redistributable	Explicit external-reference report or failure; no false self-contained export
T38	Adapter loses resource cost or visibility	Disclose loss and accept or block; no preserved automation claim
T39	Transclusion cycle, private endpoint, teleport through region	Bounded authorized rendering; specified endpoint transitions only
T40	Concurrent edges exceed relationship cardinality	One consistent accepted state; reject conflicting edit
T41	Reference workload with simulations/exports	Report p95/p99/errors; no live starvation; correct limits
T42	Restore pending prompt, dedupe records and outbox	Consistent fenced authority; no repeated roll/spend or unauthorized delivery
T43	Rounding plus unknown/divide-by-zero input	Explicit rounding; fail invalid checkpoint without NaN/ partial writes
T44	Regenerate publication under newer renderer/theme	New artifact ID; old promised artifact retained/verifiable
T45	Novices author item, conditional effect, choice and lore	Report completion/errors/abandonment; no hidden creative defaults
T46	Reference deleted entity or withdrawn package	Tombstone/retained lock or explicit unavailable result; never substitute by name
T47	Responder enumerates candidate hidden values against a delivered context token and public receipt	No match is possible; no unkeyed digest over unreadable content and no omitted-event count is delivered
T48	Cut over one character of a two-character campaign; legacy edit during an engine action	Cutover is scope-wide and a per-character cutover is rejected; no engine action reads a legacy-authoritative value
T49	Actor holds two occurrences of one item revision and invokes the second	Only the named source occurrence's resources change; an invocation without a source occurrence is schema-invalid
T50	Operator resolves an action manually; a player attempts a manual adjustment of another player's resource	Outcome records adjudicator and reason and is labelled manual in every projection; the player's attempt returns OPERATION_FORBIDDEN and changes nothing

ID	Setup / adversarial action	Required observable result
T51	Operator reorders initiative while a reaction slot is being offered	Slot restarts under the new order; recorded decliners are not offered again; no candidate is offered twice in one slot
T52	Invoker withdraws one PROPOSED action; another passes its precommit deadline	Both become CANCELLED without cost or checkpoint; the open-proposal admission count is released
T53	Oversized request body; definition over a declared structural bound; budget exceeded in a later checkpoint	413 PAYLOAD_TOO_LARGE; 422 BUDGET_EXCEEDED with nothing written; 422 BUDGET_EXCEEDED with SUSPENDED status and earlier checkpoints intact
T54	Server clock steps forward, or the service is down, past an open decision's deadline; later the clock steps back	The decision is not expired or answered and its action becomes SUSPENDED for operator review; the backward step reopens no expired decision
T55	Token moves while an action is pending, on a map without declared spatial mechanics	The move commits as representation state, triggers no mechanics and is not refused as SCOPE_BUSY
T56	Token moves while an action is pending, on a map whose geometry profile declares spatial mechanics	The move is refused with SCOPE_BUSY and changes nothing

Concurrency tests require competing transactions and process restart/failure injection, not mocks that always serialize. Permission tests cover every transport and error shape. Property tests cover operand ordering, grant multiplicity and incremental/full equivalence. Golden fixtures pin expected outputs and locks. The evaluator must ship its applicable conformance fixtures so later versions demonstrate compatibility.

22. Delivery sequence and evidence gates

The full target remains. A small complete proof now precedes broad kernel expansion. Each package has an accountable role; the team assigns people and effort estimates before scheduling. Dates cannot be inferred from theoretical design.

Gate	Deliverable / accountable role	Exit evidence
G0	Spec audit / architecture lead and security reviewer	Independent review of F01–F13; no unresolved high-risk contradiction in proposed slice
G1	Existing-character adapter; magic item; three authoring modes; resource/effect / VTT integration lead	Applicable T01–T16, T22–T26, T31–T32, T47–T50, T52–T55; sheet/token agreement; no live cutover
G2	Time, choices, grants, reactions, contrasting ruleset / rules lead	T13–T18, T28–T34, T43, T51; non-D&D boundary and full/incremental equivalence
G3	Locks, imports, updates/rollback, canonical export / content platform lead	T19–T21, T36–T38, T42, T46
G4	Complete advertised first-ruleset mechanics; progression/NPCs / rules and product leads	Every advertised mechanic classified automatic, human or manual; choice/grant/undo matrix
G5	World, relationships, mentions and spatial mechanics / world/VTT lead	T27, T39–T40, T56; secrecy across graph/render surfaces

Gate	Deliverable / accountable role	Exit evidence
G6	Publishing and wider adapters / publishing lead	T35, T37–T38, T44; accessibility and rights evidence
G7	Load, isolation, recovery and observability / operations lead	T04–T13, T25–T26, T41–T42 on deployment-like topology
G8	Optional intelligence/executable extensions / feature lead and security reviewer	Separate profile review; unchanged command/trust/replay boundaries

G1 uses bounded real primitives, which may evolve through versioned contracts when evidence reveals defects. “Final-system primitive” is not an acceptance criterion. The slice includes an existing-character conflict and hidden-effect fixture, not only a new empty character. It proves its own coverage, not the full progression/world graph.

G1/G4 product evaluation uses at least eight target users across novice and experienced authors, each attempting the four T45 tasks without an instructor choosing their mechanics. Report completion, time, mistaken interpretations, unnecessary questions and abandonment. Proposed gate: at least 80% successful task completions, zero unnoticed creative defaults in inspected outcomes, and an explicit resolution for recurrent confusion. This is formative evidence, not a market-size or population-level proof.

Before implementation authorization, provide the chosen phase's concrete schema/migration/API diff, dependency choices, executable fixture plan, rollback boundary and acceptance owner. New architecture decisions found during implementation return to a versioned design revision. This document does not authorize production cutover, new paid services or third-party scripting.

23. Requirements, finding closure and proof obligations

Requirement	Authoritative contract	Acceptance IDs
CE-IDENTITY-01	Scoped typed identities; §4	T03, T27, T40, T46, T49
CE-MIGRATE-01	Sole authority and fenced migration; §5	T01–T03, T21, T42, T48
CE-TYPE-01	Canonical types and bounded AST; §6	T14–T16, T26, T43
CE-EFFECT-01	Fixed stages, explicit stacking; §7	T14–T16, T28
CE-COMMAND-01	Authentication/durable idempotency; §8	T04–T07, T25
CE-ACTION-01	Atomic checkpoints and random records; §9	T04–T06, T12–T13, T52, T55–T56
CE-DECISION-01	Bound authorized decisions; §10	T09–T12, T32, T51, T54
CE-EVENT-01	Durable order/outbox/recovery; §§8–10	T06, T08, T42
CE-TIME-01	Logical clocks/cadence; §11	T17–T18
CE-VERSION-01	Exact closure and migration; §12	T19–T21, T42, T44, T46
CE-GRANT-01	Sourced multiplicity/compensation; §13	T28–T30
CE-POLICY-01	Policy on all commands/outputs; §14	T09, T22–T25, T36–T37, T47
CE-GUIDE-01	Bounded interview/no creative defaults; §15	T31–T33, T35, T45
CE-WORLD-01	Scoped graph/geometry; §16	T27, T39–T40, T56
CE-INTEROP-01	Authorized import/honest export; §17	T07, T36–T38

Requirement	Authoritative contract	Acceptance IDs
CE-BUDGET-01	Limits/measured workload; §18	T13, T26, T41, T53
CE-RENDER-01	Safe presentation/artifact identities; §19	T22–T24, T39, T44
CE-AUDIT-01	Linked provenance/restricted telemetry; §20	T06, T22, T25, T42
CE-MANUAL-01	Recorded, labelled manual adjudication without extra privileges; §2	T50
CE-RULESET-01	Ruleset-namespaced clocks/events and contrasting-ruleset boundary; §§2, 11	T34

Finding	Response	Status
F01	§5 field authority, ruleset target, migration and rollback	Contract addressed; migration proof pending
F02	§§8–10 receipts, checkpoints, randomness and outbox	Contract addressed; crash/concurrency proof pending
F03	§§9–10 action states, bound decisions and reactions	Contract addressed; interleaving proof pending
F04	§§6–7, 11 algebra, types, clocks and cadence	Contract addressed; evaluator proof pending
F05	§12 exact execution closure and reproduction levels	Contract addressed; restore/update proof pending
F06	§14 all-surface current policy and explicit declassification	Contract addressed; information-flow proof pending
F07	§18 deterministic work/import budgets and safe profiles	Contract addressed; isolation/load proof pending
F08	§§4, 16 templates, occurrences, facts and representations	Contract addressed; cross-campaign fixtures pending
F09	§13 grants and conflict-aware compensation	Contract addressed; progression/undo proof pending
F10	§15 bounded language, interview and quality axes	Contract addressed; language/UX evidence pending
F11	Profiles and namespaced logical clocks	Contract addressed; contrasting ruleset fixture pending
F12	§22 earlier integration/usability evidence gates	Plan addressed; staffing/benchmark evidence pending
F13	§§1, 3, 23, 25 authority, traceability and confidence	Document addressed; independent consistency review pending

G0 finding	v1.1.1 response	Acceptance
G0-01	§§6, 8, 10, 14 context tokens, public receipts, output boundary	T47
G0-02	§5 scope-level migration authority	T48
G0-03	§§4, 8 source occurrence and tagged occurrence references	T49
G0-04	§10 suspension at the mandatory deadline; reissue is an operator command	T11
G0-05	§8 command ID is a correlation identifier outside the request digest	T06, T07

G0 finding	v1.1.1 response	Acceptance
G0-06	§8 expected scope version per operation	T04, T09–T12
G0-07	§§9, 12 authority epoch vs internal fencing epochs; restore increments the authority epoch	T05, T06, T42
G0-08	§§5, 14 `controls` and `operates` relations and the VTT default grants	T09, T24
G0-09	§7 adjustment bound policy per resource	T02, T04
G0-10	§6 numeric formulas with core functions; Predicate schema	T14, T15, T43
G0-12	§5 inert `legacyMetadata` component	T01
G0-14	§2 manual play requirement CE-MANUAL-01	T50
G0-15	§10 fixed reaction candidate order per slot	T51 (G2)
G0-16	§9 PROPOSED → CANCELLED on withdrawal or precommit expiry	T52
G0-17	§§17–18 capacity and budget responses	T53
G0-18	§10 time authority, heartbeat watermark, forward-clock anomaly	T54
G0-19	§23 requirement CE-RULESET-01	T34 (G2)
G0-20	§10 schema-aligned decision fields; UTC timestamps	Schema checks
G0-11	Registry addendum A1 approved by the product owner as a companion wire contract	Addendum verification
G0-13	§§9, 16 movement is gated only where spatial mechanics are declared	T55 (G1), T56 (G5)

The next auditor should attack these invariants: committed work cannot execute twice; each scope has one authoritative state; a roll cannot be cancelled into another draw; one grant's removal preserves independent grants; secrets cross only declared disclosure boundaries; pins resolve exact behavior or fail explicitly; unsupported work cannot use an untyped bypass; and the compiler never implicitly chooses mechanical meaning for the author.

Tradeoffs remain visible: one active root mechanical action per scope; operator resolution for suspended mandatory decisions; no unrestricted deterministic prose promise; later profiles for fixed-point formulas and third-party code; no recall of already disclosed information; replay contingent on retaining permitted artifacts; unmeasured performance targets; and a required fresh inventory of the existing VTT before implementation. These bound the promise without claiming universal coverage.

24. Worked conformance traces

Shared charge. R has current=1 at scope version 40. A and B both expect version 40 and spend one charge on activation. A commits version 41, R=0 and the action gate. B changes nothing and receives a conflict/busy result. A's same-key retry returns its existing operation. B must refresh and submit a new command after A finishes; it then fails for insufficient resource if R is still zero.

Post-hit flame. The attack's hit/roll checkpoint is committed. The optional flame child references that hit and asks its authorized controller whether to spend a charge. Acceptance rechecks context and atomically debits one charge, records 2d6 outcomes and applies its declared damage. Repeated answers return that receipt. Declining preserves the attack and spends no flame charge. Later failure cannot erase either already committed checkpoint.

Private curse. A restricted effect contributes -1 defense. A reviewed rule permits the final value to be public, while source ID/name, private decision and source link remain absent from API, cache, export and socket projection. Without that declassification rule the final value is withheld. View As does not override owner-private restrictions.

Two grant sources. Species G1 and class G2 grant the same proficiency key. It is effective with two contributions. Revoking G2 preserves G1 and proficiency; removing the last source removes it only if no override/other contribution remains. No destructive proficiency-array removal is generated.

Harmless flames. The author chooses equipment then visual/light behavior, maximum three uses and an explicit recovery event. No damage component exists. Switching to damage invalidates relevant decisions and asks required amount/timing/targets. Equivalent Structured decisions yield equivalent executable meaning.

Pinned upgrade. Preview compares exact A/Evaluator1 against B/Evaluator2, reports changed semantics and excess charges, and requests a named reconciliation. Activation waits for quiescence and commits a new exact lock. Historical actions retain their original lock/outcomes; current state uses the approved replacement.

25. Source register and provenance

R01, R05, R06, R09, R14, R15, R21 and R26 were checked against primary sources during the preceding audit on 23 September 2026. R30–R31 were checked for this revision. Other R entries are inherited from v1.0 and are not claimed as independently revalidated here. C entries remain community friction evidence. §§4–24 are Roll Drauf design decisions, not requirements imposed by competitors.

- R01 — Foundry activities: https://github.com/foundryvtt/dnd5e/wiki/Activities
- R02 — Foundry effects: https://github.com/foundryvtt/dnd5e/wiki/Active-Effects
- R03 — Foundry effect guide: https://github.com/foundryvtt/dnd5e/wiki/Active-Effect-Guide
- R04 — Foundry advancement: https://github.com/foundryvtt/dnd5e/wiki/Advancement
- R05 — Foundry area behavior: https://github.com/foundryvtt/dnd5e/wiki/Activity-Behaviors
- R06 — DMV architecture/source: https://github.com/Orcpub/orcpub
- R07 — DiceCloud community slot guide: https://github.com/NappingPiglet/dicecloud-v2-guide/blob/master/creating-your-first-character/character-creation-101.md
- R08 — DiceCloud community action guide: https://github.com/NappingPiglet/dicecloud-v2-guide/blob/master/creating-your-first-character/the-stats-tab.md
- R09 — Roll20 homebrew builder: https://blog.roll20.net/posts/dd-homebrew-character-builder-is-here/
- R10 — Roll20 homebrew features: https://blog.roll20.net/posts/dd-homebrew-features-you-need-to-try-on-roll20/
- R11 — Roll20 updates: https://blog.roll20.net/posts/what-youve-missed-in-dd/
- R12 — Roll20 species/features: https://blog.roll20.net/posts/create-homebrew-dd-species-features/
- R13 — D&D Beyond roadmap: https://www.dndbeyond.com/posts/2132-d-d-beyonds-2026-development-roadmap
- R14 — D&D Beyond mid-year update: https://www.dndbeyond.com/posts/2223-mid-year-update-d-d-beyonds-2026-development
- R15 — Fantasy Grounds effects: https://fantasygroundsunity.atlassian.net/wiki/spaces/FGCP/pages/996642031/5E%2BEffects%2Bfor%2BAdvanced%2BAutomation
- R16 — World Anvil templates: https://www.worldanvil.com/learn/article-templates
- R17 — World Anvil template use: https://www.worldanvil.com/learn/article-guides/article-templates

- R18 — World Anvil settlements: https://www.worldanvil.com/learn/article-templates/settlement
- R19 — World Anvil characters: https://www.worldanvil.com/learn/article-templates/character
- R20 — Kanka mentions: https://docs.kanka.io/en/latest/features/mentions.html
- R21 — Kanka permissions: https://docs.kanka.io/en/latest/guides/testing-permissions.html
- R22 — Kanka property kits: https://docs.kanka.io/en/latest/entries/attribute-templates.html
- R23 — Kanka plugin library: https://docs.kanka.io/en/latest/plugins/plugins.html
- R24 — LegendKeeper: https://www.legendkeeper.com/features/
- R25 — Homebrewery changelog: https://github.com/naturalcrit/homebrewery/blob/master/changelog.md
- R26 — Homebrewery releases: https://github.com/naturalcrit/homebrewery/releases
- R27 — Monsterbrew announcements: https://www.monsterbrew.app/announcements
- R28 — Tetra-Cube statblocks: https://tetra-cube.com/dnd/dnd-statblock.html
- R29 — Roll20 attunement: https://blog.roll20.net/posts/dd-attunement-is-automatic-on-roll20/
- R30 — JSON Canonicalization Scheme: https://www.rfc-editor.org/info/rfc8785/
- R31 — JSON Schema 2020-12 core: https://json-schema.org/draft/2020-12/json-schema-core
- C01 — Foundry experience: https://www.reddit.com/r/FoundryVTT/comments/1dwzijd/
- C02 — Foundry accessibility: https://www.reddit.com/r/FoundryVTT/comments/1q0et78/homebrew_easeaccessability/
- C03 — Foundry automation: https://www.reddit.com/r/FoundryVTT/comments/1jdkfe1/
- C04 — Roll20 class help: https://www.reddit.com/r/Roll20/comments/1tlgf28/custom_class_builder_help/
- C05 — Roll20 reusable content: https://www.reddit.com/r/Roll20/comments/1ocekn0/dd_2024_character_builder_now_available_custom/
- C06 — D&D Beyond usability: https://www.reddit.com/r/onednd/comments/1otn3tv/

Original: Roll Drauf Creation Engine v1.0, 22 September 2026, 41 pages; SHA-256 `7f6c275ee19933f869672b05a0a3588a83b8556e02a62638e161f9eddac537e3`. Source: https://discord.com/channels/1328724663257268264/1551867598876971048/1551975805146374146

The second adversarial audit (G0) ran on 24 September 2026; v1.1.1 to v1.1.4 adopt all of its closures, and G0 passed that day. G1 implementation is underway; its recorded evidence and remaining work are summarized in §30. This reading edition does not repeat the G0 gate.

26. Architecture research incorporated after the approved baseline

Status: informative research. This section retains the research from the withdrawn draft. It does not reopen G0, select new dependencies, or change the approved G1 scope. References to earlier sections describe lessons or proposed changes, not amendments to their active contracts. The comparator sample is qualitative; it does not establish which defects are statistically most common.

This section is informative. The architecture research used a targeted qualitative sample of primary documentation, original architecture descriptions, maintainer release notes, and first-hand upstream issue reports. It does not establish a statistical ranking of the most frequent defects. Priorities reflect relevance and possible impact on Roll Drauf. No competitor bug was reproduced locally.

Evidence classes: **A** = primary technical specification, inspected source, or maintainer-recorded fix; **B** = primary product behavior or declared design without corresponding implementation verification; **I** = first-hand issue report whose reproduction or root cause has not been independently established here; **C** = secondary/community material; **D** = unsupported marketing. A closed issue is historical evidence, not a claim of a current defect. Applying a comparator's lessons to this platform is a design inference. Neither a familiar name nor a shared JSON shape proves equivalent execution semantics.

Comparable	Evidence and useful lesson	Boundary of inference
Foundry D&D5e	A; distinct activities and advancement support explicit activation and construction contracts [S01–S02]	Does not prove Roll Drauf checkpoint, rollback, or server-authority semantics
Foundry PF2e	A; typed declarative elements, ChoiceSet, and explicit grant reevaluation/deletion behavior [S03]	PF2e's selectors and stacking rules are not our generic core; declarative content still needs bounds
Dungeon Master's Vault / OrcPub	A for the original-author architecture description; hierarchical choices derive character attributes through modifiers [S04]	Character construction does not make consumed/transferred gameplay state reversible
DiceCloud	B; primary README corroborates value provenance and equipment-dependent recalculation [S05]	Its internal graph algorithm, library rules, and concurrency behavior were not inspected; inherited community guides remain C
Kanka	B; member-view permission testing with owner-private restrictions [S06]	Its documented editable member view differs from Roll Drauf's explicitly read-only intersection model
Homebrewery	A; documented Legacy/V3 render evolution and output compatibility concerns [S07]	Render equivalence does not establish exact publication bytes or authorization
Temporal	A; deterministic durable workflows, version-aware resumption, and historical replay checks [S08]	An architectural analogue, not a selected dependency; database effects and game semantics remain our responsibility
CEL	A; host-provided data, constrained expressions, and explicit complexity obligations [S09–S10]	Not selected as the evaluator: exact rationals, units, staged reads, unknown values, fuel, and provenance require our conformance proof

Foundry's token documentation distinguishes scene-specific token positions from linked resource data [S27]. The single authoritative presence chosen here is a Roll Drauf design decision, not an assertion that Foundry implements that model.

Primary entries S01–S27 are linked in §26.1. The earlier market/workflow references R01–R31 and community entries C01–C06 remain a historical register with their checking status. Roll20, D&D Beyond, Fantasy Grounds, World Anvil, LegendKeeper, and statblock tools remain product research context; their inherited descriptions are not used here to assert unverified current capabilities or implementation contracts.

The following evidence directly informs this revision. Issue reports are not treated as verified root-cause analyses:

Failure family	Evidence	Contract response
A logical resource cost applies twice	Historical first-hand Foundry report #1239 [S11]	Cost identity proposal in §28.2, retaining §§7–9 guarantees; T57
Shared tokens disagree on conditions	Foundry #7470 reports linked-token falling conflicts across positions/scenes; closed when checked [S15]	Accepted spatial direction and proposal in §28.1; T59
Separate creatures share a cadence key	Foundry #7483 reports region behavior for unlinked tokens; closed as not planned; suggested cause unconfirmed [S16]	Instance identity plus explicit cadence buckets; §11, T71
Derived HP or rules edition applied incorrectly	Maintainer notes record fixes #5785 and #5787 [S17]	Recompute from source inputs and exact locks; §§6–7, 12, T60, T62

Failure family	Evidence	Contract response
A secondary view exposes an inaccessible link	Maintainer notes record fix #5840 [S17]	Current-policy projection on every output; §14, T63
Transport order mistaken for durable delivery	Socket.IO delivery contract and outbox guidance [S12–S13]	Durable receipts, atomic outbox, deduplication and recovery; §§8–10
ORM helper commits early or misses nested JSON changes	SQLAlchemy documentation plus selected local code observations [S14, S19]	Approved checkpoint ownership; proposed extra evidence in §§27, 28.2; T58, T61
Declarative content hides unbounded work	CEL language/cost specification [S10]	Approved §18 bounds; separate retry refinement in §28.4; T64, T74

Original clean-architecture and ports-and-adapters descriptions support isolating domain rules behind explicit interfaces [S23–S24]. Their application here is §27's dependency and side-effect guidance, not a mandate for a particular number of classes, services, or layers. Hypothesis is already approved for G1; Import Linter is an optional enforcement tool [S25–S26].

The market-gap thesis remains a hypothesis: expressive mechanics may be useful when authors can reach them through a clear interview. It MUST be tested through the representative creation tasks in §22. Competitor roadmaps cannot establish a launch deadline, staffing estimate, or permission to bypass correctness gates.

26.1 Research source register

- S01 — Foundry D&D5e Activities. Evidence: technical primary.
- S02 — Foundry D&D5e Advancement. Evidence: technical primary.
- S03 — PF2e rule elements. Evidence: technical primary.
- S04 — Dungeon Master's Vault architecture. Evidence: technical primary.
- S05 — DiceCloud project README. Evidence: product primary.
- S06 — Kanka permission testing. Evidence: product primary.
- S07 — Homebrewery renderer changelog. Evidence: technical primary.
- S08 — Temporal safe deployments. Evidence: technical primary.
- S09 — CEL overview. Evidence: technical primary.
- S10 — CEL language and cost semantics. Evidence: technical primary.
- S11 — Foundry issue 1239: consumption report. Evidence: issue report.
- S12 — Socket.IO delivery guarantees. Evidence: technical primary.
- S13 — AWS transactional outbox. Evidence: technical primary.
- S14 — SQLAlchemy session basics. Evidence: technical primary.
- S15 — Foundry issue 7470: linked-token falling. Evidence: issue report.
- S16 — Foundry issue 7483: region cadence report. Evidence: issue report.
- S17 — Foundry D&D5e release 5.1.0. Evidence: technical primary.
- S18 — OWASP authorization guidance. Evidence: technical primary.
- S19 — SQLAlchemy JSON type behavior. Evidence: technical primary.
- S20 — RFC 8785 canonical JSON. Evidence: technical primary.
- S21 — PostgreSQL transaction isolation. Evidence: technical primary.
- S22 — OWASP XSS prevention. Evidence: technical primary.
- S23 — Robert C. Martin: Clean Architecture. Evidence: technical primary.
- S24 — Alistair Cockburn: Hexagonal Architecture. Evidence: technical primary.
- S25 — Hypothesis stateful testing. Evidence: technical primary.

- S26 — Import Linter dependency contracts. Evidence: technical primary.
- S27 — Foundry token documentation. Evidence: technical primary.

27. Clean coding and the implementation already underway

Status: compatible guidance and proposed additional evidence. The framework-free core, adapters, shared command boundary, PostgreSQL unit of work, and dependency choices are already described in the approved G1 plan and A1. They are not new architecture proposals. WP1–WP4 implement parts of them. The recommendations below explain how the research supports those boundaries. More specific new checks remain mapped proposals in §29; they do not silently add G1 exit criteria or invalidate completed work.

27.1 Domain boundaries and dependency direction

The implementation MUST separate deterministic domain semantics from framework and external side effects. The approved G1 plan already places the Creation Engine inside the existing application, with explicit persistence, transport, and legacy adapters. This preserves the integration boundary and supports independent core testing; extraction to another process requires a versioned decision showing how checkpoint atomicity, latency, operations, and authorization remain satisfied. Neither Temporal nor CEL is selected by analogy alone.

Responsibility	Owns	Prohibited dependency or behavior
Domain values and evaluation	Typed identities, exact quantities, AST interpretation, effect algebra, sourced grant projection, transition validation	Flask/request globals, SQLAlchemy entities, sockets, network/filesystem access, ambient clock/randomness, transaction control
Application command coordination	Authenticated command context, admission, frame/decision lifecycle, complete checkpoint plans, unit-of-work and retry orchestration	Delegating partial commits to helpers; accepting a frontend-calculated outcome as authority
Policy and authorized projections	Current audience and entitlement inputs, field restrictions, declassification, safe receipt/explanation/view data	Treating an old cached audience or imported ownership declaration as current authority
Authoring/compiler	Question graph, typed answers, invalidation, pure registered adapters, semantic draft commands	An alternate mechanical evaluator, hidden creative defaults, or persistent game-state effects
Persistence and external adapters	Scoped consistency, locks/fences, receipts/ outbox, ORM mapping, HTTP/socket translation, legacy routing	Introducing new game rules or returning unfiltered ORM objects through transports
Packages and publication	Inert parsing/preview, exact lock resolution, adapter loss reports, authorized semantic rendering	Activation during parsing; rules execution in templates; renderer-created game values

Source dependencies point toward domain/application contracts. Domain functions receive immutable typed snapshots and explicit deterministic inputs and return values, conflicts, unresolved results, or proposed transitions. They do not obtain a Session, Request, global service locator, or arbitrary callback capable of mutation. Adapter interfaces expose the narrow capabilities required by the use case. Read-only previews and renderers MUST NOT receive a write-capable unit of work. An import-boundary check enforces these restrictions for the engine package.

The server evaluator is canonical. A client preview may share pure code or use a separately conformed implementation, but must identify its input/lock and satisfy the same semantic fixtures. A template, legacy form, or GUI convenience calculation cannot become a second authoritative rules implementation. Differences in persistence layout, UI toolkit, or language do not justify different mechanics.

27.2 Cohesion, types, and explicit effects

Single responsibility is enforced around reasons for change: a reducer calculates, a coordinator commits, a projection filters, and a renderer presents. Helpers used within a checkpoint MUST NOT commit, emit success, allocate independent grant records, or draw ambient randomness. Before entering the pure evaluator, the application materializes the authorized computation snapshot under its consistency boundary; lazy ORM reads cannot escape into pure evaluation or rendering.

Use distinct domain types for the approved Definition/Revision/Instance references, Quantity/unit, draft version, scope version, and worker fence. Future spatial work adds its own Scene/Representation references and presence version without changing existing wire types silently. Wire syntax validation is followed by relational and state-transition validation. Reusing a generic dictionary or integer type at an external boundary does not remove the obligation to discriminate these values before domain execution. Constructors/parsers do not silently coerce strings to numbers, zero to missing, or unknown to false.

Published revision values and evaluator inputs are immutable. Mutable persistence entities are mapped at the adapter boundary; returned projections are detached values that cannot write back to storage. In particular, a cached total is never a base input, and a view read is never a lifecycle command. Persistence tests reload committed JSON and related records in a fresh session/connection instead of trusting the in-memory object that was just mutated.

Prefer composition through registered components and functions over a hierarchy that hardcodes D&D creature categories in the core. Extensions are versioned registry entries with schema, purity, cost, policy, and conformance contracts. Runtime discovery or monkey-patching of ordinary content as privileged code is prohibited. Interfaces should be small enough that a read-only consumer cannot accidentally acquire unrelated write capabilities.

Do not turn stylistic similarity into a shared abstraction when semantics differ: a draft edit, published revision, and runtime adjustment have different authority and lifecycle. Share actual mechanical semantics and explicit value types. Arbitrary function-length targets, class counts, and universal repositories are not conformance criteria. Domain names such as resolve_decision, revoke_grant_source, project_for_audience, and commit_checkpoint are preferred to vague mode-driven helpers that conceal authority in boolean flags. Exceptions or result objects must distinguish domain conflict, unresolved input, policy denial, and operational interruption; public diagnostics are separately projected.

27.3 Checkpoint and adapter acceptance

The coordinator's unit of work MUST be visible in code review. A reviewer must be able to identify its read set, policy/fence validation, all planned costs/changes, logical fuel, random-outcome handling, receipt identity, event/outbox writes, and actual commit point. No notification is a committed success until that unit commits. A serialization failure retries the whole attempt within the approved §18 contract (the separate retry proposal is §28.4), while ambiguous results recover the existing receipt. Every external side effect has an explicit owner and retry contract.

REST, sockets, legacy commands, and registered background jobs translate to the same semantic application handlers. Transport authentication remains at the boundary; domain logic receives the server-established principal and explicit capabilities, never a principal claimed by the payload. An adapter may preserve a legacy response shape only by transforming the permitted projection. It cannot bypass current policy, resource checks, presence authority, or the lock to remain compatible.

The persistence handoff MUST state the chosen backend/version, isolation or equivalent locking argument, uniqueness/fencing constraints, lock order, policy-race strategy, session lifecycle, migration fence, and ambiguous-commit recovery. The scope-wide serial-order argument includes account/policy changes that affect commands, even when those records live outside the mechanical scope table. A transaction around writes alone is insufficient. Tests on an in-memory substitute do not establish this argument for another database.

27.4 Verification and review gates

Each affected invariant needs targeted evidence as the remaining work packages are implemented. Pure examples test exact algebra, unknowns, grant multiplicity and deterministic provenance. Generated cases compare full and incremental evaluation and cold/warm cache logical fuel. Stateful sequences exercise grant/revoke, equip/transform, spend/recover, placement/transfer, retry, and conflicting undo against a simpler reference model. Expected results come from the specification or an independent model, not a copy of the implementation under test.

Transaction tests use the selected database, independent connections/workers, and explicit barriers to force races and crash windows. They test policy revocation, competing receipt claims, presence uniqueness, shared resources, lease fencing, outbox replay, operational retry exhaustion, and restore. Compatibility tests compare semantic outcomes across transports. Projection tests cover each authorized role and output surface, including old receipts and delayed rendering after revocation. A registered renderer/adapter has fixtures for its claimed capability and losses.

The engine boundary MUST have automated dependency, static-type or equivalent contract, formatter/linter, schema, and applicable acceptance checks in CI. Tool choice is an implementation decision. Adoption should scope checks to the new boundary and touched integration paths; unrelated repository-wide cleanup is not required. Coverage percentages, snapshots alone, or linters cannot substitute for the critical negative cases. Hypothesis is approved and already used; Import Linter is optional; implementing equivalent checks with existing tooling is acceptable.

Every implementation review identifies the affected requirement and acceptance IDs, transaction owner, authority inputs, expected retry/crash behavior, public projection, lock/schema compatibility, and remaining test gaps. Refactors preserve observable semantics and fixture results; changes to mechanics, provenance ordering, logical fuel, or geometry require a new compatible/versioned lock and explicit migration behavior.

27.5 Current implementation choices retained

The approved G1 design uses the existing application's isolated engine package, PostgreSQL/SQLAlchemy persistence, RFC 8785 canonicalization, JSON Schema validation, property tests with Hypothesis, and the approved Redis delivery integration. These choices are not reopened by this consolidation. The remaining public policy/transport work is WP6, legacy integration is WP7, authoring is WP8, and milestone evidence is WP9. Future service extraction, a replacement evaluator, or changed deployment topology would be separate changes. CEL and Temporal remain research comparables; Import Linter remains an optional way to enforce already-required dependency direction.

28. Research changes and their contract boundaries

Status: scoped change proposals, except the already accepted spatial product direction. Sections 1–24, core wire 1.1.3, registry A1 and the approved G1 plan continue to govern current implementation. The terms below express the proposed behavior if the affected change is adopted. They are not runtime commands registered by this document. Each proposal has an affected gate, compatibility analysis and future evidence in §29.

28.1 Accepted spatial direction and its boundary with G1

The accepted direction is one authoritative spatial presence per EntityInstance in a RuntimeScope. A presence record is either unplaced or placed in one scene/frame with one position and footprint. Several live views may project that location; reference markers may show other locations without mechanical authority. Independent creatures have separate instances. Shared resources across multiple simultaneous physical bodies require a later explicit capability.

This direction belongs to spatial mechanics, principally G5. The approved E4 distinction remains: moving a token on a map without declared spatial mechanics is an ungated representation edit (T55); moving a mechanical token during another active action on a map with declared spatial mechanics is refused (T56). Under the proposed

presence model, an ordinary map marker remains a reference representation. Dragging it does not place or move the authoritative presence, including when the same occurrence also has a presence on another map. Mechanical placement, movement and transfer are explicit operations subject to the gate. This reconciles the accepted direction with E4; it does not turn all token movement into a mechanical command.

The G1 adapter still maps token and sheet resources to the occurrence and leaves ordinary x/y representation edits alone as A1 specifies. No presence table, four-command rollout or ExecutionLock shape change is added to the current G1 work packages by this document.

The following detailed spatial design is retained for the affected gate's design work. New SceneRef/ RepresentationRef shapes, coordinate/geometry types, authority rules and migration behavior require a versioned registry/lock compatibility design before they become an implementation contract. Existing commands and immutable schema IDs must not be redefined.

28.1.1 Authoritative presence and representations

Within a RuntimeScope, an EntityInstance MUST have at most one authoritative spatial presence. The occurrence owns a persistent SpatialPresence record with a monotonically increasing version and a placement tagged `unplaced` or `placed`. A placed value identifies one RuntimeScene, coordinate frame, exact 2D position, occupied footprint, and locked geometry-profile digest. An unplaced value has no mechanical coordinates. Presence versions do not reset when unplaced, re-placed, restored, or displayed through another token.

A RuntimeScene belongs to exactly one RuntimeScope. Its registered coordinate frame declares units, exact scale, and any supported transforms. Its mechanical geometry and access policy are versioned inputs to commands. Two scenes are not assumed to share coordinates because their map images overlap or have the same filename. Cross-scene distance, sight, auras, and attacks are unsupported under this proposal; a later capability must define the connection explicitly.

A representation is either a **live view**, whose mechanical scene/position/footprint is projected from the occurrence's presence, or a **reference view**, whose display layout is independent and nonmechanical. Multiple live views of the same presence may exist, but each shows the same authoritative location in a compatible coordinate frame and contributes no additional physical body. A representation at another scene or conflicting position is a reference view. Changing a display's requested view mode cannot relocate or place an occurrence; the server validates its binding and derives any live coordinates from the existing presence. An unplaced occurrence has no live spatial view until an authorized placement commits. The UI identifies reference views to authorized users and routes attempts at mechanical manipulation to the real presence or an explicit placement/transfer workflow.

A sheet, disguise, world-map marker, prepared encounter token, hidden layer, or second browser cannot acquire authority merely by being selected, opened, moved visually, or rendered last. Reference views MUST NOT establish spatial target eligibility, occupied area, range, aura sources, region entry/exit, cadence consumption, collision, or mechanical vision. They also MUST NOT provide an alternate route to map information that the current scene/presence/policy would otherwise withhold. Operator map-preview permissions are separate read capabilities; they do not create a presence or grant player vision.

Hiding a live view is presentation behavior and does not make the creature mechanically invisible. Invisibility, footprint changes, and other mechanical state changes require the relevant rules commands. Deleting a live view removes that view only; the occurrence and presence remain. The operator can recover an authorized view of an otherwise undisplayed presence. No remaining marker is silently promoted. If the intent is to remove the creature from spatial play, use explicit unplacement or the appropriate lifecycle command.

28.1.2 Presence commands and authority

The following are candidate command payloads for future spatial registration. They are absent from the current approved wire schemas; the three core commands and all A1 operations remain unchanged. The proposal would use the mechanical envelope, expected scope version, authority epoch, lock digest, and idempotency identity. Each also carries a full InstanceRef and `expectedPresenceVersion`; matching a reference's scope to the envelope remains a semantic obligation.

Command	Required payload and state	Observable semantics
PlacePresence	Unplaced occurrence; destination SceneRef/ frame/position; registered placementRuleId; reason	Derive the footprint from authorized current mechanics, validate destination and entry semantics, and place once
MovePresence	Placed occurrence; current SceneRef/frame; mode continuous or teleport; bounded path; registered movementRuleId	Validate path start against current presence, movement/teleport authority and costs, then execute the declared windows
TransferPresence	Placed occurrence; destination in another scene of the same scope; registered transitionRuleId; reason	Commit departure/arrival location and membership changes together at the relocation checkpoint
UnplacePresence	Placed occurrence; registered transitionRuleId; reason	Explicitly remove spatial occupancy under declared exit semantics, preserving occurrence state and history

Placement and unplacement require explicit campaign operator capabilities under this proposal. A controller's movement permission permits only supported movements under the named locked rule; it does not authorize arbitrary teleportation. A transfer requires an operator transfer capability or an actor activity that expressly authorizes that scene transition. Internal child movement uses the current action frame's validated authority. An unrelated wire command cannot enter an active frame by omitting its frame binding; it receives SCOPE_BUSY. Registered operator interventions identify the current frame and invalidate affected prompts through the normal decision protocol.

The client does not supply authoritative footprint, geometry-profile identity, state versions to install, conditions to clear, or vision grants. The server derives these from the snapshot and lock. All scene, frame, profile, presence, and movement prerequisites are checked again at commit. Invalid/inaccessible destinations follow §14's disclosure rules; known accessible conflicts use PRESENCE_VERSION_CONFLICT, PRESENCE_STATE_CONFLICT, or SPATIAL_CONTEXT_REQUIRED as appropriate. UI selection is never an authorization input.

Continuous movement has a path of 2–256 exact 2D points in the current frame; the first equals the committed starting position. Teleport has exactly two points, start and destination, and never traverses intermediate regions. Scene transfer uses a destination coordinate in its destination frame and no fictitious path between unrelated frames. Footprint changes through a transformation use the same occupancy-reconciliation boundary and increment presence version; changing icon scale alone does not change occupied geometry.

28.1.3 Checkpoints, transitions, and geometry

All mechanical presence changes use the scope gate and checkpoint protocol. A scene transfer atomically replaces the occurrence's one placed location; no committed state places it in both scenes or briefly drops its state into a fresh occurrence. Its relocation checkpoint commits the new presence version, old/new region memberships, immediate declared spatial changes, receipt, events, audit linkage, and outbox together. A failed precommit transfer leaves the old presence intact. A committed transfer recovered after response loss is not repeated.

Registered spatial rules declare pre-departure and post-arrival windows, costs, region eligibility, and any exit/entry behavior. If a mandatory decision is required before relocation, the frame waits while the old location remains authoritative. Subsequent reactions can suspend the frame after relocation; the committed location is not rolled back merely because later work waits or fails. A continuous path whose mechanics can halt movement must declare supported intermediate movement checkpoints and their ordering; the remaining path is not committed ahead of the interruption. An unsupported movement/interrupt combination fails closed or is resolved manually with recorded authority.

The proposed geometry profile supports 2D circles and simple polygons, exact rational coordinates, and at most 64 vertices per polygon footprint. Polygon closure is implicit; vertices do not repeat the first point. Polygons must have nonzero area and no self-intersections; circle radii are strictly positive. Footprints use local offsets from the presence position in its frame units. Touching counts as intersection. Exact predicates avoid floating-point epsilon

choices. Intersection ordering need not persist an intersection point as a presence position. If a rule requires stopping at a point not representable in the declared coordinate type, a separately specified exact placement or explicit rounding policy is required; the engine must not introduce an implicit approximation. Elevated/3D visibility, falling physics, and other height-dependent automation require a separately versioned geometry capability; the 2D profile does not invent those semantics.

A placed footprint intersects a region when their geometry intersects. Committed continuous movement emits crossed boundaries in increasing path parameter, with equal-parameter ties ordered by region ID. Teleport emits exits for old-only memberships and entries for new-only memberships. Scene transfer emits departures in stable old-region order, then arrivals in stable new-region order; unplacement has departures only and placement has arrivals only. Events identify the occurrence and transition/checkpoint, not a display token. Multiple views cannot multiply event emission. Any profile-specific pre-window effect that changes the movement plan requires revalidation before the affected checkpoint.

Area selectors deduplicate matching live views by full InstanceRef before costs, effects, or per-target cadence are computed. Independent occurrences of one definition remain separate targets. Spatial application identity includes source application, target InstanceRef, and the declared membership/cadence identity. Auras originate only from an authoritative placed presence. Leaving an aura removes its occupancy-derived contribution according to the rule; independent timed conditions such as poison remain until their own removal/expiry event. A reference marker on the old scene cannot keep an aura active or clear a condition.

28.1.4 Unplaced play, history, and presentation

An unplaced occurrence may still be edited, use supported nonspatial mechanics, and participate in authorized manual play. A required spatial fact while unplaced is unresolved, not zero distance, an empty target set, or permission to act from any marker. Spatial automation requiring it returns SPATIAL_CONTEXT_REQUIRED or follows an explicit human adjudication contract. Unplacement is not a player shortcut out of hazards, costs, or a pending decision; it requires its stated authority and transition rule and remains subject to the scope gate.

HP, resources, grants, timed effects, action identities, and turn/cadence buckets remain with the occurrence through placement changes. A scene transfer does not end an encounter or reset a turn by itself. Any legitimate encounter transition is an explicit locked ruleset event. Presence and scene versions used by pending decisions would be recorded internally and bound to the approved opaque contextToken; an authorized intervention invalidates affected unanswered decisions. Backup/replay retains presence history, movement classification, geometry-profile identity, and causal events. Animation, interpolation, camera state, and a viewer's selected scene never generate authoritative movement events.

A separate actual creature uses a separate EntityInstance even if its definition, appearance, or name is identical. A future multi-body capability must explicitly define shared/independent resources, conditions, perception, action economy, target identity, and cadence. The proposed model rejects attempts to create a second authoritative presence for one occurrence. It may still render an authored multi-body concept at manual quality without pretending to automate it.

28.2 Stable cost identity and transaction ownership

The approved kernel already owns atomic checkpoints, source-occurrence spending and durable idempotency. The research adds a proposed explicit cost key `(actionId, checkpointId, costId, boundedOccurrenceIndex)` to distinguish a repeated delivery from two legitimate costs on the same pool. Costs sharing a pool should be checked against aggregate availability while retaining distinct identities. Existing WP4 proofs remain evidence for their recorded scope; they are not automatically evidence for this additional cost-key convention. T57 and T58 add targeted review cases.

The proposal to distinguish a durable admission/attempt record from a committed checkpoint is a clarification of the boundary, not a requirement to change current public receipt shapes. A pending record is never proof that resources changed. Public output remains `PublicReceipt`, filtered under current permissions, without an outcome digest or omitted-event count. Receipt recovery must retain the original identity and respect current account/read access; it cannot turn a stale retry into a fresh mutation.

28.3 Deterministic provenance refinement

Full and incremental result/provenance equivalence is already required by §7. The following ordering is a refinement proposal; it is not retroactively imposed on the pinned WP2 evaluator without a compatibility comparison and version decision.

Internal provenance is a typed acyclic graph with stable semantic node identities. Each contribution identifies target binding, stage, source kind, source occurrence/revision where applicable, contribution ID, and change index. Unordered contributor lists are sorted lexicographically by that tuple, using fixed stage ordinals and ASCII identifier order; absent optional source fields sort before present fields. Semantic priority still determines SET selection before this presentation order. Source identities are assigned by originating commands or definitions, never allocated by a cache or renderer. Node IDs are digests of their canonical semantic identity under the input/lock digest. Equal-valued winning sources all remain recorded.

Canonical provenance cannot depend on hash-map iteration, database row order, worker completion order, locale, or cache warmth. Explicit ordered sequences keep their declared order. Public explanations first apply current policy and permitted disclosure, then order the remaining permitted nodes; they expose neither omitted nodes nor their original indexes/counts. Internal provenance digests are not automatically public. The pinned evaluator defines a reference evaluation order and its logical work charge under the proposal in §28.4.

The internal semantic input/lock digests in that proposal remain private whenever they cover unreadable facts. Clients receive permitted projected explanations and, when binding an answer to private context, the approved opaque `contextToken`. No proposed presence or provenance field reintroduces the withdrawn public `contextDigest` contract.

28.4 Logical fuel and operational retry proposal

The approved §18 limits and locked evaluator semantics remain the current contract. The research recommends distinguishing deterministic logical charges from the amount of work caused by database failures. The five-attempt recovery episode, durable attempt ledger and cache/reference-charge rules below are new detailed proposals. They require an impact review of the existing evaluator and kernel, a lock/version strategy where behavior changes, and focused acceptance evidence; they are not an instruction to rewrite WP2/WP4 or defer WP5 now.

Each AST literal, read, arithmetic/boolean operation and conditional evaluation charges one logical operation when evaluated by the pinned reference evaluator; registered compound functions charge their internal primitive and input-iteration work. Query visits, trigger firings and event creation have explicit counters and cannot hide work inside one function call. A registered function without a bounded cost contract is unavailable. The reference evaluator fixes dependency traversal, branch evaluation, memoization within one checkpoint, and contributor ordering. Failed work within an evaluation and internal function retries are charged; branches that the reference semantics do not evaluate are not charged merely because another implementation speculates on them.

Semantic fuel and operational attempts are separate budgets. The root frame retains the cumulative fuel charges of its committed checkpoints and children. A successful checkpoint atomically records its charge with its outcome; duplicate delivery or receipt lookup charges it no second time. An uncommitted evaluation checks its work against both checkpoint limits and the root's remaining committed allowance. A deterministic exceedance aborts that checkpoint as BUDGET_EXCEEDED and follows §9. A worker restart or new child cannot reset committed root counters. A checkpoint resumed under the same supported inputs and lock must have the same logical charge as a fresh reference evaluation.

Incremental evaluation and result caching may reduce actual CPU work, but MUST NOT change logical fuel admission or acceptance. A cache hit carries or reconstructs the reference charge and deterministic provenance for the same inputs. Implementations separately measure actual CPU, memory, and query work and enforce operational limits. A warmed cache cannot make an otherwise over-budget rules action valid; a cold cache cannot make a logically valid action fail as BUDGET_EXCEEDED. Operational resource exhaustion may still interrupt work and return the operational status without inventing a rules result.

The proposed automatic database retry policy permits at most five execution attempts per checkpoint recovery episode, counting the first attempt. Durable attempt claims bind operation/checkpoint, episode, attempt ordinal, and worker fence before expensive work begins. They survive rollback and worker replacement and are distinct from game-state mutation. A confirmed database abort may consume another attempt; an ambiguous commit must first resolve the durable outcome. Each attempt has configured finite time/memory limits and admission controls. Client resubmission, transport switching, or worker restart does not create a new episode or refill its budget.

Exhaustion returns OPERATION_RETRY_EXHAUSTED with the existing operation status. Before activation the pending operation has no committed game effect or acquired active gate; after activation the frame is suspended and retains its gate. An authorized operator can start another bounded recovery episode for that same operation after operational recovery; the action retains its lock, committed outcomes, semantic fuel, and audit history. Mechanical expected-version conflicts are reported as conflicts, not repeatedly retried with silently refreshed inputs. The deployment handoff records concrete operational time/memory/admission settings and verifies that automated recovery cannot repeat indefinitely through an external job supervisor.

The references to §9 above concern checkpoint history. `OPERATION_RETRY_EXHAUSTED`, episode records and their proposed handling are not added to the currently approved API merely by appearing here. Client-facing statuses and an error/registry addition must be reviewed together with the implementation change. Existing decisions still obey the approved clock-anomaly handling, internal lease fence versus authority epoch, and decision-specific expected-version rules.

28.5 Migration, persistence and verification refinements

Scope-level migration authority from E1 remains mandatory: no engine action may read a legacy-authoritative mechanical dependency, and there is no per-character cutover inside one active scope. The future presence migration adds explicit resolution of conflicting token locations; it must not choose the last viewed token or infer a location from a timestamp. This is additional spatial migration design, not a replacement for the existing scope migration protocol.

Fresh-session JSON reloads, same-command transport comparisons, cache comparisons and architecture import checks are proposed evidence for existing or refined invariants. They should be assigned to the affected work package after checking what its tests already establish. Inert `legacyMetadata`, explicit adjustment-bound policy, typed Predicates, UTC timestamps, exact source occurrences, and current-policy projection remain approved behavior, not optional research suggestions.

29. Acceptance reconciliation and change disposition

T01–T56 and the 20 approved requirement rows in §§21 and 23 retain their exact approved meanings. Their historical catalogue status is not a current test dashboard; implementation evidence is recorded by work package in the G1 program. G0 approval and the WP1–WP4 records are retained. The tests below are proposed additions, not executed results or automatic new G1 exit criteria.

The withdrawn draft had reused T47–T56 for unrelated research scenarios. The corrected mapping shifts its T47–T64 to T57–T74. Always qualify an old citation as `withdrawn-v1.2:Txx` before translating it; an unqualified approved T49 still means source-occurrence isolation, for example. Original research RA01–RA10 map to T57–T66.

ID	Setup / adversarial action	Required observable result
T57	One activity has ammunition and charge costs; retry through another transport; also two legitimate costs against one pool	Each declared logical cost applies once; distinct cost IDs remain distinct; aggregate availability is checked and the original receipt is recovered

ID	Setup / adversarial action	Required observable result
T58	Fault after a legacy adapter prepares a resource change but before the full receipt/outbox unit commits	No partial checkpoint persists; helpers cannot commit independently; postcommit retry recovers the full prior outcome
T59	One occurrence has a live dungeon view and conflicting reference views on other scenes; compare independent copies	Only the canonical presence establishes geometry/ events/vision; references do not toggle conditions; independent occurrences retain independent state
T60	Transform, equip, and rebuild repeatedly; permute unordered contributors and compare full/incremental results	No derived value is reapplied as a base; values, unknowns, conflicts and canonical provenance match without persistent projection side effects
T61	Edit a nested draft answer, commit and reload in a new session; repeat with precommit failure	Answer and descendant invalidation persist together on success; neither persists on failure
T62	Similar content names under distinct rules editions/ evaluators; restore with an exact required artifact missing	Use the selected exact closure or report unavailable; no edition/name/latest fallback; migration reconciliation remains explicit
T63	Request a private contribution through every output surface, warm another audience cache, then revoke access	Only declared declassification is exposed; old receipts/ delayed jobs use current policy; no private names, IDs, links, or omitted counts escape
T64	Compound function and child actions approach root fuel limits; force aborts, worker restart, and five automatic attempts	Internal work and committed root charges persist correctly; operational retries do not refill episodes or change logical fuel; exhaustion has no partial game effect
T65	Canonicalize Unicode-key and numeric-boundary vectors across runtimes; parse duplicate-key source bytes	Identical valid canonical bytes/hashes; reject duplicate keys before collapse and invalid mechanical values; zero/false/missing/unknown stay distinct
T66	Restart a pending spatial decision; expire worker lease; accept one valid answer then deliver late/stale/revoked responses	Same action and committed rolls; only one eligible current-context answer; stale worker fence and old presence contexts cannot commit
T67	Concurrent placement, move or transfer commands use the same scope and presence version	At most one conflicting new transition commits; no second presence or lost update; duplicates return their receipt
T68	Move, hide, delete or select a reference/live display; open another browser scene	No unintended relocation, invisibility, aura, vision or cadence change; deleting a view preserves its occurrence and presence
T69	Transfer between scenes across regions with a pre-departure decision, post-arrival reaction, and lost response	Old presence remains until relocation commit; departure/arrival reconcile once; later suspension preserves destination and prior state; active unrelated transfer is busy
T70	Unplace and re-place a poisoned occurrence with spent resource and cadence; attempt spatial action while unplaced	Explicit authority and exit/entry rules; poison/spend/ cadence persist; required geometry is unresolved and returns spatial guidance or authorized manual resolution
T71	Two live views of one occurrence and two independent occurrences of one definition overlap the same region	Deduplicate views by InstanceRef; independent targets retain distinct per-target cadence; scene transfer never resets logical buckets
T72	Introduce a framework/ORM/network import or ambient randomness/commit into the domain boundary	Automated architecture/purity checks reject the violation; pure domain fixtures run without application or database startup

ID	Setup / adversarial action	Required observable result
T73	Equivalent authorized semantic commands arrive via REST, sockets and legacy adapter with transport differences	Same semantic handler, digest identity, costs and current-policy outcome; transport-specific payload cannot bypass the command boundary
T74	Evaluate the same snapshot with cold cache, warm cache and incremental optimization at logical fuel boundaries	Same reference charge and acceptance/result/ provenance; actual CPU may differ; operational interruption is not misreported as deterministic budget exhaustion

29.1 Proposed requirement mappings

These six IDs are proposed additions. They do not replace CE-MANUAL-01 or CE-RULESET-01, and they do not change the approved requirement-to-case mappings in §23. References here identify the proposal's clauses and tests rather than declaring the whole section normative for current G1.

Requirement	Proposed contract	Acceptance IDs
CE-PRESENCE-01	Occurrence-owned presence with E4-compatible representation edits; §28.1	T59, T66–T71
CE-PROVENANCE-01	Stable contribution identities/order and pure projection; §§27, 28.3	T60, T65, T74
CE-RETRY-01	Logical fuel separate from bounded operational recovery; §28.4	T64, T74
CE-ARCH-01	Explicit dependency, type and side-effect evidence; §27	T58, T60–T61, T72–T73
CE-PERSIST-01	Whole-checkpoint ownership and real-backend/fresh-session evidence; §§27, 28.2	T57–T58, T61, T67
CE-TEST-01	Additional lock, policy, recovery and adapter evidence; §§27–28	T57–T74

29.2 Disposition by change, without restarting G1

Change	Authority / compatibility	Affected work and adoption condition
Approved E1–E4 corrections and registry A1	Retained in full, with existing schemas, IDs and gates	Continue current G1 plan; no new approval or audit needed merely to retain them
Clean-code boundaries and checkpoint ownership	Already supported by approved plan/A1; additional fixtures are proposals	Apply existing boundaries in remaining WPs; add new checks through ordinary scoped review
Single authoritative presence	Product direction accepted; detailed contract not yet approved for implementation	G5 spatial design; preserve E4 and ordinary G1 representation editing
Spatial commands and geometry/lock fields	Breaking additions cannot be smuggled into frozen wire 1.1.3	Design a versioned registry and lock extension at the spatial gate; migration/ compatibility review then
Explicit cost identity	Refines existing spending guarantees	Kernel impact review; retain valid WP4 results and add only necessary fixtures

Change	Authority / compatibility	Affected work and adoption condition
Canonical provenance tuple	More specific than current equivalence requirement	Evaluator compatibility comparison and version decision before changing locked output
Five-attempt episodes and cache-independent logical charge	New detailed budget/recovery behavior	Separate evaluator/kernel change proposal; not a prerequisite for the current WP5 contract
Research findings already covered by existing tests	Evidence can be reused only for demonstrated behavior	Map tests to both IDs with their scope; do not rerun or relabel all work as unproven

30. Active contract, implementation status and next action

The project remains in G1 Deploy. G0 passed on 24 September 2026. The approved specification is v1.1.4, the core wire contract is 1.1.3, and registry A1 supplies 24 additional commands. The core supplies three, for 27 approved command shapes. A documentation revision number does not bump any of these protocol identifiers.

The approved plan records WP1 (canonical types), WP2 (evaluator), WP3 (content store and RDL subset) and WP4 (transactional kernel) as implemented and committed. It records a WP4 full regression run of 1,067 passing tests and a final focused engine run of 435 passing tests. Those are historical results with their own scope. At reconciliation, HEAD was `66b6fc447c5bf5b15886aeec559961a2b68e30b9`; WP5 decision/deadline work was present as uncommitted changes. This documentation task neither executes those engine tests nor certifies the unfinished changes.

The next implementation action is to finish and verify WP5 against v1.1.4/core 1.1.3/A1, then continue WP6 policy/public transport, WP7 legacy integration, WP8 three authoring modes, and WP9 milestone evidence and user study. The user-approved locales remain German, English and French. The RDL subset, flame sword/private curse fixture, applicability rulings and approved dependencies remain as recorded in D1–D8. No live cutover is authorized by this reconciliation.

Future research deltas receive a narrow review when their affected package/gate is ready. They do not invalidate the passed G0 or require repeating G1 discovery and architecture. A discovered implementation defect against the current contract should be fixed against that contract; it is not a reason to replace it with an unapproved draft.

The repository's `docs/creation-engine/README.md` is the entry point. It names the approved specification, wire schemas, approval record and implementation program. This v1.2 document is their reconciled reading edition plus an explicitly bounded research change register. The source manifest and verifier check that the approved body, acceptance identities, schemas and A1 remain intact. The withdrawn package is historical evidence only.